DataSynth: Reference Knowledge Graphs for Enterprise Audit Analytics through Synthetic Data Generation with Provable Statistical Properties

Michael Ivertowski
Independent Researcher
Zurich, Switzerland
https://orcid.org/0009-0008-7829-2249

April 2026

Abstract

Enterprise knowledge systems for audit analytics face a fundamental epistemological challenge: they are typically constructed from existing operational data—data that may itself contain the very errors, fraud patterns, and systematic biases the systems are meant to detect. This is analogous to assembling a jigsaw puzzle with only a partial view of the picture on the box: teams assemble fragments of transactional evidence into knowledge graphs, but lack a *ground truth* against which to verify the result. When the source data harbors systematic errors—which real-world client audits demonstrably do—the constructed knowledge inherits and amplifies those errors, potentially obscuring root causes that audit procedures seek to identify. The complexity of the problem is not merely large; it is *inter-dimensional*, spanning accounting topology, statistical distributions, temporal dynamics, regulatory standards, and organizational structure simultaneously.

We present DataSynth, an open-source framework that addresses this challenge by generating *reference knowledge graphs*: fully provenanced, multi-layer synthetic enterprise datasets where every node, edge, and property is produced from a known generative process. DataSynth operates across three layers of audit knowledge: **(i)** the *structural layer*—accounting graph topology with a comprehensive set of entity and relationship types spanning major enterprise processes; **(ii)** the *statistical layer*—distributional properties governed by log-normal mixture models, five copula families, Benford's-law compliance, and temporal dynamics calibrated against 155 real-world general ledger datasets comprising 364 million journal entries; and **(iii)** the *normative layer*—accounting standards (US GAAP, IFRS, French GAAP, German GAAP, dual-reporting), audit standards (ISA, PCAOB, SOX), COSO 2013 internal controls, and regulatory compliance rules.

We prove that recovering ground truth from observed enterprise data is combinatorially infeasible (the number of consistent accounting network configurations grows super-exponentially with journal entry complexity), and that systematic errors in source data propagate through inverse-reconstruction pipelines with high probability of remaining undetected. DataSynth circumvents this by generating data *forward* from a fully specified generative model, producing reference datasets where the complete audit trail is known by construction.

Implemented in Rust, DataSynth achieves throughput exceeding 2×10^5 journal entries per second on a single core. The framework generates end-to-end coherent data spanning journal entries, document flows, subledgers, financial statements, intercompany eliminations, tax accounting, treasury management, ESG reporting, project accounting, and OCEL 2.0 process mining event logs—all from a single declarative YAML configuration. A methodology-driven audit engine loads 12 audit methodology blueprints—spanning ISA, IIA-GIAS, PCAOB, AICPA SOC 2, and Big 4 firm-style methodologies—as YAML-defined finite state machines, producing event-sourced audit trails with per-step `judgment_level` annotations for automation-potential analysis. End-to-end financial coherence is enforced: manufacturing costs flow through WIP to COGS with standard cost variances, treasury instruments generate hedge accounting entries per ASC 815/IFRS 9, and tax provisions derive from actual pre-tax income—validated by 25+ coherence checks including a trial balance master proof. Additional capabilities include anomaly injection with over 130 subtypes and multi-stage fraud schemes, a privacy-preserving fingerprint module under ε-differential privacy, a counterfactual simulation engine with causal DAG propagation, and graph exports in PyTorch Geometric, Neo4j, and DGL formats for GNN-based audit analytics.

Experimental evaluation demonstrates that generated datasets satisfy all hard accounting invariants at 100% compliance, achieve Benford first-digit MAD scores below 0.006, and support downstream fraud detection models achieving F1 scores of 0.84 with graph neural networks—within 3% of scores obtained on real data.

The forward-generation approach does not replace the analysis of real enterprise data; rather, it provides a reference frame against which analytical tools, knowledge construction pipelines, and audit procedures can be developed, tested, and calibrated.

Keywords: reference knowledge graphs, synthetic data generation, enterprise accounting, audit analytics, ground truth, knowledge layers, Benford's law, copula models, fraud detection, differential privacy, process mining, graph neural networks, counterfactual analysis, audit methodology, finite state machines

1 Introduction

The digitization of enterprise resource planning (ERP) systems has produced vast repositories of financial transaction data that underpin modern accounting, auditing, and regulatory compliance. Machine-learning approaches to fraud detection [11, 30], continuous auditing [29], and anomaly analytics [26] have shown considerable promise, yet they face a challenge that goes deeper than mere data access.

The puzzle without a picture. Consider how most teams approach the construction of enterprise knowledge systems for audit analytics. They begin with the data that *exists*: general ledger extracts, trial balances, subledger reports, document archives. From these fragments, they attempt to reconstruct the underlying business reality—the true network of transactions, relationships, and control flows. This is fundamentally an *inverse problem*: inferring the generative process from its outputs.

The analogy to a jigsaw puzzle is precise. A competent puzzler works from the picture on the box—the ground truth—arranging pieces to match a known target. But enterprise knowledge construction proceeds *without* this picture. Teams assemble transactional fragments

into knowledge graphs, applying professional judgment and domain expertise to interpret the emergent structure. However, when internally consistent patterns emerge, distinguishing genuine correctness from the appearance of coherence in systematically biased data remains challenging. A systematically biased dataset can produce an internally coherent knowledge graph that nonetheless misrepresents aspects of the underlying reality.

Systematic errors and the illusion of coherence. This problem is not hypothetical. Real-world client audits contain systematic errors that may remain undetected due to inherent information constraints in the available data [10, 23]. A vendor master file with duplicates introduces phantom entities into a knowledge graph. An intercompany matching process with rounding errors propagates balance discrepancies through consolidation. A payroll system with misconfigured tax tables produces systematically incorrect withholdings that remain undetected because the errors are consistent within the system's own logic.

The complexity of detecting such errors is not merely large—it is *inter-dimensional*. Errors span accounting topology (wrong accounts, missing references), statistical distributions (Benford violations, unusual amount patterns), temporal dynamics (wrong periods, suspicious timing), normative rules (GAAP violations, control bypasses), and organizational structure (segregation-of-duties violations, approval chain anomalies). No single dimension of analysis can capture the full error landscape, and the interactions between dimensions create a combinatorial complexity that is difficult to address comprehensively through any single analytical approach.

The need for reference knowledge. We argue that the solution lies in reversing the direction of knowledge construction. Rather than attempting to recover ground truth from observed data (the inverse problem), we generate synthetic data *forward* from a fully specified generative model, producing *reference knowledge graphs* where the complete audit trail is known by construction. These reference graphs serve three purposes:

1. **Ground truth for algorithm validation.** Any audit analytics algorithm—whether rule-based, statistical, or ML-based—can be tested against data where every anomaly, every relationship, and every control violation is known a priori.

2. **Benchmark for knowledge system evaluation.** Knowledge graph construction pipelines can be evaluated by comparing their output against the reference graph, measuring precision, recall, and structural fidelity.

3. **Training data with provenance.** ML models trained on reference data inherit the three layers of knowledge embedded in the generative process, producing more robust and interpretable audit analytics.

Contributions. We present DataSynth, a comprehensive open-source framework that generates reference knowledge graphs for enterprise audit analytics. Our contributions are:

- A **formal analysis of the ground truth problem** (Section 2) showing that recovering ground truth from observed enterprise data is combinatorially infeasible, that systematic

errors propagate with high probability through inverse reconstruction, and that forward generation from a known model is the principled solution.

- A **three-layer knowledge architecture** (Sections 4 and 7) spanning structural knowledge (entity types and relationship types detailed in Section 7), statistical knowledge (Benford-compliant distributions, copula-based dependencies), and normative knowledge (five accounting frameworks, ISA/PCAOB/SOX audit standards, COSO 2013 internal controls).
- A **layered generation architecture** (Section 4) that separates domain models, statistical distributions, business-process generators covering major enterprise processes, output sinks, and evaluation into composable modules.
- A **statistically grounded sampling engine** (Section 5) featuring log-normal mixture models, five copula families for dependency modeling, and first- and second-digit Benford compliance with provable MAD bounds.
- A **multi-process generation pipeline** (Section 6) producing coherent data spanning P2P, O2C, R2R, S2C, HR, manufacturing, tax, treasury, ESG, and project accounting processes with three-way matching, intercompany elimination, and period-close procedures.
- A **reference knowledge graph construction module** (Section 7) that produces heterogeneous graphs with full provenance in PyTorch Geometric, Neo4j, and DGL formats, enabling GNN-based audit analytics with known ground truth.
- A **multi-stage anomaly injection framework** (Section 8) with over 130 anomaly subtypes across five categories, progressive fraud schemes, and ground-truth labels suitable for supervised and semi-supervised learning.
- A **privacy-preserving fingerprint module** (Section 9) enabling generation of synthetic data that mirrors real-world characteristics under ε-differential privacy without exposing individual records.
- A **counterfactual simulation engine** (Section 10) with causal DAG propagation for what-if analysis of financial process dependencies.
- A **comprehensive evaluation suite** (Section 11) with coherence validators, statistical tests, ML-readiness metrics, and an auto-tuning engine.

Empirical grounding. Our design is informed by a large-scale empirical study of 155 real-world ISO 21378:2019–compliant general ledger datasets comprising 364 million journal entries with 2.4 billion line items across all industry sectors [17]. This analysis reveals the distributional characteristics of production accounting data—e.g., that 92.9% of entries have 2–9 line items with extreme concentration at 2-line entries (60.7%), and that chart-of-accounts complexity spans 76 to 2,536 accounts—which directly calibrate DataSynth's generation models (Section 12.2).

Scope and audience. DataSynth targets three communities: (a) ML researchers needing realistic labeled accounting datasets with known ground truth for fraud detection and anomaly analytics, (b) auditors and regulators seeking reference knowledge graphs for validating continuous auditing tools and knowledge construction pipelines, and (c) ERP vendors requiring high-volume test data for system integration and stress testing.

The remainder of this paper is organized as follows. Section 2 formalizes the ground truth problem and introduces the three layers of audit knowledge. Section 3 surveys related work. Section 4 presents the system architecture. Section 5 formalizes the statistical foundations. Section 6 details the generation pipeline. Section 7 describes reference knowledge graph construction. Section 8 covers the anomaly injection framework. Section 9 discusses the privacy-preserving fingerprint module. Section 10 introduces the counterfactual simulation engine. Section 11 presents the evaluation suite. Section 12 reports experimental results. Section 13 concludes.

2 The Ground Truth Problem in Enterprise Knowledge Systems

This section formalizes the fundamental challenge that motivates DataSynth: the inherent limitations of recovering complete ground truth from observed enterprise data alone, and how this motivates the forward-generation approach.

2.1 The Inverse Problem of Knowledge Recovery

Enterprise knowledge systems for audit analytics are typically built by a process we term *inverse knowledge recovery*: given observed data D (journal entries, document flows, master records), reconstruct the true generative process $\mathcal{G}$ that produced it. We formalize this problem and show it is fundamentally ill-posed.

Definition 2.1 (Enterprise Data Observation)**.** *An* enterprise data observation *is a tuple* $D = (J, M, R, T)$ *where:*

- $J = \{j_1, \ldots, j_N\}$ *is a set of journal entries, each with line items* $j_k = \{l_{k,1}, \ldots, l_{k,n_k}\}$ *where each line item specifies an account, amount, and debit/credit indicator;*
- M *is a set of master data records (vendors, customers, materials, employees);*
- R *is a set of document references linking records across tables;*
- T *is a set of timestamps and period assignments.*

Definition 2.2 (Accounting Network)**.** *Given a journal entry* j *with* n *credit line items and* m *debit line items, an* accounting network *maps each credit-to-debit flow as a directed edge in a graph* $G = (V, E)$ *where nodes* V *are GL accounts and edges* E *carry monetary amounts. The edge construction requires a* matching *of credit amounts to debit amounts such that flows are conserved.*

The central difficulty is that the mapping from journal entries to accounting network edges is not unique. For a single journal entry with n credit and m debit line items, the number of valid matchings is given by the surjective function count.

Proposition 2.3 (Combinatorial Infeasibility of Ground Truth Recovery)**.** *Let j be a journal entry with n credit line items and m debit line items. The number of distinct accounting network edge configurations consistent with j is bounded below by:*

$$\mathcal{A}(n,m) = \sum_{k=0}^{m} (-1)^{m-k} \binom{m}{k} k^n \tag{1}$$

which corresponds to the number of surjections from the credit set to the debit set [17]. For an enterprise with N journal entries, the total configuration space is:

$$|\mathcal{S}| = \prod_{j=1}^{N} \mathcal{A}(n_j, m_j) \tag{2}$$

Proof. The edge construction problem for a single entry requires assigning each of n credit line items to at least one of m debit line items such that total flows balance. This is a surjection from the set of credits to the set of debits. The number of surjections from an n-element set to an m-element set is $m! \cdot S(n,m)$ where $S(n,m)$ is the Stirling number of the second kind [17]:

$$S(n,m) = \frac{1}{m!} \sum_{k=0}^{m} (-1)^{m-k} \binom{m}{k} k^n.$$

Multiplying through gives $\mathcal{A}(n,m) = \sum_{k=0}^{m} (-1)^{m-k} \binom{m}{k} k^n$, which is Equation (1). Since journal entries are independent, the total configuration space is the product over all entries.

For concrete scale: in the 155 real-world datasets analyzed by Ivertowski et al. [17], only 60.7% of entries have $n = m = 1$ (trivial 1:1 matching). The remaining 39.3% require non-trivial matching. For a modest entry with $n = 4$ credits and $m = 3$ debits, $\mathcal{A}(4,3) = 36$. An enterprise with 10^5 such non-trivial entries has $|\mathcal{S}| > 36^{10^5} \approx 10^{155{,}630}$ possible configurations—a number that dwarfs the number of atoms in the observable universe ($\sim 10^{80}$). □

Remark 2.4. *Proposition 2.3 counts discrete credit-to-debit* assignments. *When monetary amounts must also be split across edges, the configuration space becomes continuous and uncountably infinite, making the discrete count a lower bound on the true ambiguity.*

Corollary 2.5. *For any enterprise dataset with a non-trivial fraction of multi-line journal entries, enumerating all consistent accounting network configurations is computationally infeasible. Consequently, without additional information beyond the journal entries themselves, distinguishing the true configuration from the exponentially many alternatives is not possible by exhaustive search.*

2.2 Systematic Error Propagation

The combinatorial infeasibility of Proposition 2.3 means that knowledge recovery must rely on heuristics and assumptions. We now show that when the source data contains systematic errors, these propagate through any reconstruction pipeline with high probability.

Definition 2.6 (Systematic Error)**.** *A* systematic error *in enterprise data is a consistent deviation from the true state that affects multiple records in the same direction or pattern. Unlike*

random errors, systematic errors do not cancel under aggregation and may be invisible to internal consistency checks.

Observation 2.7 (Internal Consistency Does Not Imply Correctness)**.** *A dataset D may satisfy all standard accounting invariants—balanced entries (Equation (12)), the accounting equation ($A = L + E$), subledger reconciliation, document chain integrity—while containing systematic errors in amounts, entity classifications, period assignments, or relationship structures. Internal consistency is necessary but not sufficient for correctness.*

Proposition 2.8 (Error Persistence in Multi-Stage Processes)**.** *Consider an enterprise process with k sequential stages (e.g., PO → GR → Invoice → Payment → GL Posting → Financial Statement, giving $k = 6$). Let each stage have an independent probability ε_d of detecting and correcting a systematic error introduced at an earlier stage. The probability that the error reaches the final stage undetected is:*

$$\mathbb{P}[\textit{undetected}] = (1 - \varepsilon_d)^{k-1} \tag{3}$$

For typical detection rates $\varepsilon_d \in [0.01, 0.05]$ and $k = 6$:

ε_d	k	$\mathbb{P}[\textit{undetected}]$
0.01	*6*	*95.1%*
0.02	*6*	*90.4%*
0.05	*6*	*77.4%*

Even with a 5% per-stage detection rate, over three-quarters of systematic errors survive the full processing chain.

Proof. Each stage provides an independent opportunity to detect the error. The error survives stage i with probability $(1 - \varepsilon_d)$. Since stages are independent, the probability of surviving all $k - 1$ stages after introduction is $(1 - \varepsilon_d)^{k-1}$. □

Remark 2.9. *The independence assumption in Proposition 2.8 is optimistic: in practice, detection capabilities are often correlated across stages (e.g., a subtle classification error that evades one stage is likely to evade similar checks at subsequent stages). The actual persistence probability for systematic errors may therefore be higher than $(1 - \varepsilon_d)^{k-1}$, making this a conservative estimate of the problem's severity.*

2.3 Inter-Dimensional Complexity

The difficulty of the ground truth problem is compounded by the *inter-dimensional* nature of enterprise data errors. We identify five dimensions along which errors manifest, and show that their interactions create a complexity class that exceeds any single-dimension analysis.

Definition 2.10 (Knowledge Dimensions)**.** *Enterprise audit knowledge spans five orthogonal dimensions:*

1. ***Topological*** *($\mathcal{D}_T$): Account classifications, entity relationships, document reference chains, graph connectivity.*

2. ***Statistical*** *($\mathcal{D}_S$): Amount distributions, Benford's-law compliance, correlation structures, outlier patterns.*

3. ***Temporal*** *($\mathcal{D}_\tau$): Posting dates, period assignments, processing lags, seasonal patterns, regime changes.*

4. ***Normative*** *($\mathcal{D}_N$): Accounting standards compliance, audit requirements, internal control effectiveness, regulatory adherence.*

5. ***Organizational*** *($\mathcal{D}_O$): Approval hierarchies, segregation of duties, departmental structure, intercompany relationships.*

Observation 2.11 (Cross-Dimensional Error Amplification)**.** *Let e be a systematic error in dimension $\mathcal{D}_i$ that affects n_i records. If the error induces secondary effects in d additional dimensions, each affecting a multiplicative factor $\alpha_j \geq 1$ records, then the total error footprint is:*

$$|e^*| = n_i \cdot \prod_{j=1}^{d} \alpha_j \tag{4}$$

For typical enterprise processes where $d \geq 2$ and $\alpha_j \geq 2$, a single-source error affecting 100 records can propagate to ≥ 400 affected data points across dimensions.

Illustrative example. Consider a vendor master data error (topological dimension) that assigns incorrect GL account codes to a vendor. This produces: (a) statistical anomalies in the affected accounts' distributions ($\alpha_S \geq 1$), (b) temporal artifacts from misclassified period assignments ($\alpha_\tau \geq 1$), and (c) normative violations in subledger reconciliation ($\alpha_N \geq 1$). Each propagation multiplies the affected record count. The product follows from the independence of propagation paths across dimensions. The multiplicative model provides a useful upper-bound heuristic for estimating cross-dimensional impact, though actual propagation depends on the specific error type and process dependencies.

Remark 2.12. *The inter-dimensional nature of errors explains why single-dimension audit analytics—Benford tests alone, or control testing alone, or process mining alone—systematically underperform. Effective audit analytics must operate across all five dimensions simultaneously, which requires training data that embeds knowledge across all five dimensions. This is precisely what* DataSynth*'s reference knowledge graphs provide.*

2.4 Three Layers of Audit Knowledge

Having established that inverse recovery is infeasible and error-prone, we now define the *forward generation* approach through three layers of audit knowledge. DataSynth generates data that explicitly encodes all three layers, producing reference knowledge graphs with full provenance.

Definition 2.13 (Three-Layer Knowledge Model)**.** *A* reference knowledge graph $\mathcal{K}$ *comprises three layers:*

Layer 1: Structural Knowledge ($\mathcal{K}_S$). *The accounting graph topology—accounts as nodes, transaction flows as directed edges, master data entities with typed relationships, document chains with referential integrity. This layer answers* what entities exist and how they connect.

Layer 2: Statistical Knowledge ($\mathcal{K}_\Sigma$). *The distributional properties of all quantities—amount distributions conforming to Benford's law, temporal patterns (period-end spikes, intraday profiles, processing lags), cross-field correlations via copula models, economic cycle dynamics. This layer answers* what quantitative patterns characterize normal operation.

Layer 3: Normative Knowledge ($\mathcal{K}_N$). *The rules, standards, and controls that govern the enterprise—accounting framework (US GAAP, IFRS, etc.), audit standards (ISA, PCAOB), internal controls (COSO 2013 with 17 principles), regulatory requirements (SOX 302/404), and organizational policies (approval thresholds, SoD rules). This layer answers* what constraints define correct behavior.

Proposition 2.14 (Reference Knowledge Graph Completeness)**.** *Let $\mathcal{G}$ be a fully specified generative model that explicitly encodes structural rules $\mathcal{R}_S$, statistical parameters Θ_Σ, and normative constraints $\mathcal{C}_N$. A dataset $D = \mathcal{G}(seed)$ generated by $\mathcal{G}$ constitutes a reference knowledge graph that is* complete *with respect to all three layers:*

$$\mathcal{K}(D) = \mathcal{K}_S(\mathcal{R}_S) \cup \mathcal{K}_\Sigma(\Theta_\Sigma) \cup \mathcal{K}_N(\mathcal{C}_N) \tag{5}$$

Furthermore, for any predicate ϕ over D that is decidable given $\mathcal{G}$, the ground truth value $\phi(D)$ is computable in time polynomial in $|D|$.

Justification. By construction, every record in D is produced by a deterministic function of the seed and the generative model $\mathcal{G}$. The structural rules $\mathcal{R}_S$ determine which entities exist and how they relate (Layer 1 completeness). The statistical parameters Θ_Σ determine all distributional properties (Layer 2 completeness). The normative constraints $\mathcal{C}_N$ are explicitly encoded in the generation process, so every violation or conformance is known (Layer 3 completeness). This is a direct consequence of forward generation from a fully specified model: since the model is known, any question about the data that depends only on the model's specification is answerable by inspecting the model. □

2.5 Forward Generation vs. Inverse Recovery

We summarize the contrast between the two paradigms:

Property	Inverse Recovery	Forward Generation
Ground truth	Partially recoverable via domain expertise	Known by construction
Systematic errors	May be inherited if undetected	Absent (or injected with labels)
Completeness	Rich in context but limited by observability	Complete across all three layers
Complexity	Exponential (Proposition 2.3)	Linear in output size
Verifiability	Verification constrained by data availability	Every fact is traceable to the model
Anomaly labels	Typically requires manual annotation	Generated with full provenance
Reproducibility	Depends on data access and agreements	Deterministic from seed

The forward generation paradigm does not replace the need to analyze real enterprise data—it provides the *reference frame* against which analysis tools, knowledge construction pipelines, and audit procedures can be developed, tested, and calibrated before deployment on real data. DataSynth implements this paradigm as a practical tool, as detailed in the following sections.

3 Related Work

We position DataSynth at the intersection of five research streams: enterprise knowledge systems and audit analytics, synthetic tabular data generation, accounting-domain data synthesis, anomaly injection for fraud detection, and privacy-preserving data sharing.

3.1 Enterprise Knowledge Systems for Audit

The construction of knowledge graphs from enterprise data has gained increasing attention in audit analytics. Existing approaches follow the inverse recovery paradigm described in Section 2.1: they extract entities and relationships from operational data to build audit-relevant knowledge structures.

Guo et al. [10] demonstrate that accounting graph topology—degree distributions, clustering coefficients, and community structure—can discriminate between normal and fraudulent bookkeeping patterns, validating the information content of structural knowledge (Layer 1 in our framework). Boersma [5] applies complex network analysis to audit data, modeling interfirm relationships and transaction patterns as multi-layer graphs. Liang et al. [20] develop pattern recognition and anomaly detection methods directly on bookkeeping graph representations.

These approaches generally rely on available operational data as their starting point and do not separately establish that the input faithfully represents the underlying business reality. As discussed in Section 2.2, when systematic errors are present—precisely the situation where audit analytics is most critical—this reliance can limit the reliability of the resulting knowledge structures. DataSynth complements these approaches by providing reference knowledge graphs with known ground truth, enabling rigorous evaluation of knowledge construction methods.

3.2 General-Purpose Tabular Data Generation

Deep generative models have become the dominant paradigm for synthetic tabular data. CTGAN [32] introduced mode-specific normalization and conditional training for mixed-type columns. TVAE [32] applies variational autoencoders with the same preprocessing. Subsequent work has refined these approaches: CopulaGAN [24] combines copula transformations with GAN training, REaLTabFormer [28] leverages autoregressive transformers for relational tables, and TabDDPM [19] adapts diffusion models to tabular settings.

While these methods excel at preserving marginal distributions and pairwise correlations, the enterprise accounting domain imposes additional requirements beyond what general-purpose tabular generators address: cross-table referential integrity, temporal ordering constraints, and domain-specific algebraic invariants (e.g., balanced entries). Furthermore, audit analytics requires a normative knowledge layer—accounting standards, controls, and regulatory compliance—that is beyond the scope of general-purpose generators.

3.3 Accounting-Domain Data Synthesis

Domain-specific synthetic data for accounting has received comparatively little attention. Schreyer et al. [27] propose a federated learning framework for training anomaly detection models on journal entry data across audit engagements, addressing data-sharing constraints but not the generation of synthetic reference data. BankSim [21] focuses narrowly on payment transactions for fraud research.

These systems were designed to address specific aspects of the synthetic data challenge and serve their intended purposes effectively. However, they do not produce a comprehensive reference knowledge graph with provenance across all three layers, which is what evaluating end-to-end knowledge construction pipelines requires.

3.4 Accounting Network Construction

The representation of double-entry bookkeeping as directed graphs dates back to Ijiri [13], who established that journal entries have a natural graph interpretation: accounts are nodes, and debit/credit flows form directed edges. Fellingham [8] provides a modern algebraic treatment of double-entry systems. Arya et al. [1] study the problem of inferring individual transactions from aggregated financial statements, highlighting the combinatorial complexity of reconstructing edge-level flows from node-level balances—a result closely related to our Proposition 2.3.

Ivertowski et al. [17] present a hardware-accelerated approach for constructing accounting networks from general ledger data, analyzing 155 real-world ISO 21378:2019–compliant datasets comprising approximately 364 million journal entries with 2.4 billion line items. Their analysis reveals the distributional characteristics and combinatorial complexity that directly inform DataSynth's generation model and motivate the forward-generation approach.

While these approaches focus on *analyzing* real accounting data as graphs, none address the complementary problem of *generating* synthetic accounting data that produces reference knowledge graphs with known ground truth. DataSynth bridges this gap.

3.5 Anomaly Injection and Fraud Simulation

Fraud detection benchmarks typically rely on post-hoc label injection or manual scenario creation. The IEEE-CIS fraud dataset [12] provides real transaction labels but lacks accounting context. Pourhabibi et al. [25] survey fraud detection methods and note the scarcity of labeled multi-dimensional accounting fraud data. Dbouk and Jamali [6] inject statistical anomalies into time series but do not model multi-stage fraud schemes that evolve over time.

DataSynth introduces a *scheme-based* injection model where fraud evolves through state-machine stages (e.g., vendor kickback schemes that escalate over months), producing temporally correlated, multi-transaction anomaly patterns with ground-truth labels spanning all five knowledge dimensions (Definition 2.10).

3.6 Privacy-Preserving Data Sharing

Differential privacy [7] provides formal guarantees for statistical queries. DPGAN [31] and PATE-GAN [18] train generative models under DP constraints but are computationally expensive and may degrade statistical fidelity. DataSynth's fingerprint module takes a different approach: it extracts distributional summaries under ε-DP, producing a compact archive from which a fully configured generator can be instantiated, cleanly separating the privacy boundary from the generation process.

3.7 Process Mining and Event Logs

Object-centric event logs (OCEL 2.0) [9] generalize traditional single-case-ID event logs by associating events with multiple typed objects. While tools such as PM4Py [4] support OCEL 2.0 import/export, generating realistic synthetic OCEL 2.0 logs with consistent underlying business data has not been addressed. DataSynth produces OCEL 2.0 event logs that are traceable to the same journal entries, document flows, and master-data records that constitute the rest of the reference knowledge graph.

3.8 Summary of Gaps

Table 1 summarizes feature coverage. No existing system produces a complete reference knowledge graph spanning all three knowledge layers with full provenance.

4 System Architecture

DataSynth is implemented as a Rust workspace comprising 17 crates organized into five architectural layers (Figure 1). The layered design enforces separation of concerns and maps directly onto the three-layer knowledge model (Definition 2.13): domain models and statistical primitives in the foundation layer encode Layer 2 (statistical) knowledge, generators in the process layer produce Layer 1 (structural) knowledge, and standards and evaluation modules encode Layer 3 (normative) knowledge.

Table 1: Feature comparison with related systems. Columns: **KG** = reference knowledge graph, **3L** = three knowledge layers, **Bal** = balanced entries, **Ben** = Benford compliance, **Doc** = document-flow chains, **Sub** = subledger coherence, **Std** = accounting/audit standards, **Anom** = anomaly injection with labels, **Graph** = graph export for GNNs.

System	KG	3L	Bal	Ben	Doc	Sub	Std	Anom	Graph
CTGAN / TVAE	–	–	–	–	–	–	–	–	–
SDV / CopulaGAN	–	–	–	–	✓*	–	–	–	–
REaLTabFormer	–	–	–	–	✓	–	–	–	–
BankSim	–	–	–	–	–	–	–	✓	–
Schreyer et al.	–	–	–	–	–	–	–	–	–
Guo et al.	–	✓*	–	–	–	–	–	–	✓*
DataSynth	✓	✓	✓	✓	✓	✓	✓	✓	✓

*Partial support.

```
Layer 4 Analytics  Export datasynth-eval | datasynth-graph |
datasynth-graph-export datasynth-fingerprint
Layer 3 Domain Generators datasynth-generators | datasynth-banking
| datasynth-ocpm datasynth-standards | datasynth-audit-fsm
datasynth-audit-optimizer
Layer 2 Orchestration  Output datasynth-runtime | datasynth-output |
datasynth-config
Layer 1 Foundation datasynth-core (models, distributions, resource guards)
datasynth-test-utils
```

Figure 1: Layered crate architecture of DataSynth (17 crates). Dependency arrows point downward; each layer depends only on layers below it.

4.1 Layered Crate Organization

Layer 1: Foundation. `datasynth-core` defines domain model types spanning accounting (journal entries, ACDOCA), master data (vendors, customers, materials, fixed assets, employees), document flows (P2P, O2C), sourcing (S2C), financial reporting, HR/payroll, manufacturing, sales, bank reconciliation, intercompany, subledgers, FX/close, tax accounting, treasury management, ESG/sustainability, project accounting, audit (ISA 600 group audit), audit documentation and methodology, business combinations, pensions, and relationship/graph types. Statistical distribution samplers include Benford, copula (5 families), mixture, Pareto, Weibull, Beta, zero-inflated, conditional, drift, and temporal models. Resource-management primitives (memory guard, CPU monitor, disk guard, degradation controller) ensure stable operation under load. All financial values use `rust_decimal::Decimal` for exact arithmetic, eliminating IEEE 754 rounding errors.

Layer 2: Orchestration. `datasynth-runtime` houses the `GenerationOrchestrator` that coordinates the full generation workflow. `datasynth-config` provides schema validation, industry presets (manufacturing, retail, financial services, healthcare, technology), and complexity-level mappings. `datasynth-output` implements streaming sinks for CSV, JSON, Parquet, and Apache Arrow formats.

Layer 3: Domain Generators. `datasynth-generators` contains specialized generators organized by business process: P2P document flows, O2C chains, S2C sourcing, HR/payroll (including benefits enrollment), manufacturing (BOM, quality, cycle counts, inventory movements), tax accounting (multi-jurisdiction, VAT/GST, deferred tax, transfer pricing), treasury (cash positioning, forecasting, hedging, debt covenants), ESG/sustainability (GHG emissions, energy, water, waste, diversity, safety, governance), project accounting (WBS, earned value, change orders, milestones), period-close, intercompany, and anomaly injection. `datasynth-banking` adds KYC/AML-specific generation with five money laundering typologies. `datasynth-ocpm` produces OCEL 2.0 compliant event logs. `datasynth-standards` implements five accounting frameworks (US GAAP, IFRS, French GAAP/PCG, German GAAP/HGB, and dual-reporting) together with ISA, SOX, and PCAOB audit standard models.

Audit Methodology Engine. The `datasynth-audit-fsm` crate introduces a methodology-driven approach to audit data generation. Rather than producing audit artifacts through independent, stateless generators, it loads audit methodology blueprints—encoded as YAML-defined finite state machines—and walks each procedure's state machine in topological (precondition-respecting) order. Twelve built-in blueprints are provided, spanning ISA, IIA-GIAS [14], PCAOB, AICPA SOC 2, and Big 4 firm-style methodologies (Deloitte, PwC, KPMG, EY variants), enabling cross-firm benchmark comparison of audit procedure structure and coverage. Each step transition emits a deterministic audit event and dispatches to the appropriate artifact generator (materiality calculation, risk assessment, workpaper, etc.), producing both an event-sourced audit trail and the concrete typed artifacts. Steps carry a `judgment_level` classification (`data_only`, `ai_assistable`, or `human_required`), supporting analysis of automation potential across methodologies. A generation overlay mechanism separates the methodology definition ("what happens") from the simulation parameters ("how it generates"), enabling the same blueprint to produce engagements of varying thoroughness and anomaly profiles. Streaming execution via `run_engagement_streaming` emits events through a callback during FSM traversal, enabling real-time integration with downstream consumers. ISA 600 group audit procedures are modeled with component auditor instructions, reports, and multi-entity coordination, and year-over-year engagement chains support multi-period audit simulation. The companion `datasynth-audit-optimizer` crate converts blueprints into directed graphs (via `petgraph`) for shortest-path analysis and Monte Carlo simulation of engagement outcome distributions.

Layer 4: Analytics. `datasynth-eval` provides the evaluation framework (15+ coherence validators, statistical tests, ML-readiness metrics, auto-tuner). The graph crates (`datasynth-graph` and its export companion) construct and export heterogeneous reference knowledge graphs in PyTorch Geometric, Neo4j, and DGL formats with 78+ entity types and 39+ relationship edge types. `datasynth-fingerprint` implements privacy-preserving extraction and synthesis.

Layer 5: Interfaces. A CLI binary (`datasynth-data`) supports `generate`, `validate`, `init`, `info`, and `fingerprint` subcommands. A REST/gRPC/WebSocket server enables streaming generation with API-key authentication and rate limiting.

4.2 Knowledge Layer Mapping

The three knowledge layers (Definition 2.13) map to the architecture as follows:

Knowledge Layer	Crates	Artifacts
Structural ($\mathcal{K}_S$)	generators, graph, ocpm	Entity types, edges, document chains
Statistical ($\mathcal{K}_\Sigma$)	core (distributions), eval	Amount distributions, correlations, temporal
Normative ($\mathcal{K}_N$)	standards, generators (controls), audit-fsm	GAAP/IFRS rules, ISA/SOX, COSO, audit trails

Every generated record carries provenance tracing it to one or more knowledge layers, enabling downstream consumers to understand not just *what* was generated but *why*—a property that is difficult to achieve comprehensively when constructing knowledge from observed data alone.

4.3 Deterministic Reproducibility

All randomness flows from a single `u64` seed through the ChaCha8 cryptographic PRNG [3]. Each generator receives a derived seed computed as $\texttt{base\_seed} + \texttt{offset}(g)$ where $\texttt{offset}(g)$ is a fixed constant per generator type g. This guarantees:

1. **Bitwise reproducibility:** identical seed $\Rightarrow$ identical output across runs and platforms.
2. **Generator independence:** modifying one generator's parameters does not perturb another's output stream.
3. **Parallel safety:** each thread maintains a local RNG instance; no global state is shared.

Deterministic reproducibility is essential for reference knowledge graphs: the same seed always produces the same ground truth, enabling controlled experiments and reproducible benchmarks.

4.4 Collision-Free UUID Generation

Synthetic records require globally unique identifiers that are deterministic yet collision-free across generator types. DataSynth employs an FNV-1a hash-based UUID factory with a 16-byte structure:

$$\underbrace{\texttt{seed}[0{:}5]}_{\text{6 bytes}} \parallel \underbrace{\texttt{gen\_type}}_{\text{1 byte}} \parallel \underbrace{\texttt{0x4}\parallel\texttt{sub}}_{\text{1 byte}} \parallel \underbrace{\texttt{counter}}_{\text{8 bytes}} \tag{6}$$

The `gen_type` discriminator byte (39 values, `0x01`–`0x3E`) partitions the UUID space by generator, ensuring that journal-entry IDs can never collide with vendor IDs or document-flow IDs regardless of counter values.

4.5 Resource Management and Graceful Degradation

Large-scale generation runs may stress system resources. DataSynth implements a unified resource guard monitoring memory (via `/proc/self/statm` on Linux, `ps` on macOS), disk space (via `statvfs`), and CPU utilization with configurable thresholds. When thresholds are breached, the system transitions through four degradation levels:

Level	Memory	Batch Size	Features Disabled
Normal	$< 70\%$	100%	None
Reduced	70−85%	50%	Data quality injection
Minimal	85−95%	25%	Anomaly injection
Emergency	$> 95\%$	Flush	Optional fields

A 5% hysteresis band prevents oscillation between levels.

4.6 Configuration System

DataSynth uses a hierarchical YAML configuration with 30+ sections covering global settings, companies, chart of accounts, transactions, document flows, distributions, temporal patterns, anomaly injection, internal controls, intercompany, banking, accounting and audit standards, vendor networks, customer segmentation, relationship strength, cross-process links, tax, treasury, ESG, project accounting, HR, manufacturing, sales, and more.

Five industry presets (manufacturing, retail, financial services, healthcare, technology) provide complete default configurations. Three complexity levels (small ∼100 accounts, medium ∼400, large ∼2,500) control chart-of-accounts granularity and transaction volume.

Multi-GAAP Framework. DataSynth supports framework-aware generation across five accounting standards: US GAAP, IFRS, French GAAP (Plan Comptable Général), German GAAP (HGB), and dual-reporting mode. Each framework determines GL account numbering conventions, classification rules, and depreciation methods. Seven country packs (FR, JP, CN, IN, IT, ES, CA) bundle currency, tax regime, fiscal-year convention, and banking configurations. German jurisdictions support GoBD-compliant audit export, and French jurisdictions produce FEC extracts.

4.7 Streaming Pipeline and Session Management

DataSynth supports phase-aware streaming output via the `PhaseSink` trait. Three sink implementations are provided: file (JSONL), HTTP, and no-op (benchmarking).

Multi-period generation is managed by `GenerationSession`, which persists state to `.dss` checkpoint files between runs. `SessionState` tracks RNG seeds, account balance snapshots, document-ID counters, and entity counts across periods, enabling incremental generation and long-horizon simulations.

4.8 Performance

Key optimizations include `SmallVec<[T; 4]>` for line-item collections, precomputed temporal CDF tables, binary search over cumulative weights, `itoa`/`ryu` fast formatting, and `zstd` output compression. The `ParallelGenerator` trait splits the seed space deterministically across threads, achieving near-linear scaling up to 8 cores and exceeding 2×10^5 journal entries per second on a modern x86-64 core.

5 Statistical Foundations

This section formalizes the statistical machinery underlying DataSynth's data generation—the *statistical knowledge layer* ($\mathcal{K}_\Sigma$) of the reference knowledge graph. We cover Benford-compliant amount sampling, copula-based dependency modeling, mixture distributions, and temporal dynamics.

5.1 Benford's Law Compliance

Benford's law [2, 23] states that the leading digit $d \in \{1, \ldots, 9\}$ of naturally occurring numerical data follows the distribution:

$$P(D_1 = d) = \log_{10}\left(1 + \frac{1}{d}\right). \tag{7}$$

The generalization to the first two digits $d_1 d_2 \in \{10, \ldots, 99\}$ is:

$$P(D_{12} = d_1 d_2) = \log_{10}\left(1 + \frac{1}{d_1 \cdot 10 + d_2}\right). \tag{8}$$

Benford compliance is a critical property of the statistical knowledge layer: real enterprise data consistently follows this law, and violations are a well-established indicator of manipulation [23]. DataSynth implements three Benford-aware samplers:

BenfordSampler. Uses CDF-based inverse-transform sampling over the first-digit probabilities. Given a pre-computed CDF vector $F = [0.30103, 0.47712, 0.60206, \ldots, 1.0]$, a uniform draw $u \sim \mathcal{U}(0, 1)$ is mapped to digit d via binary search.

EnhancedBenfordSampler. Extends first-digit sampling to the first-two-digit distribution (Equation (8)), critical for passing second-digit tests used in forensic accounting.

BenfordDeviationSampler. Generates amounts that *intentionally violate* Benford's law for anomaly injection. Five deviation modes: RoundNumberBias, ThresholdClustering, UniformFirstDigit, DigitBias, and TrailingZeros.

Proposition 5.1 (MAD bound)**.** *Let $\hat{p}_d$ denote the empirical first-digit frequency from n samples of the BenfordSampler. The mean absolute deviation* $\text{MAD} = \frac{1}{9}\sum_{d=1}^{9} |\hat{p}_d - p_d|$ *satisfies, for large n,*

$$\mathbb{E}[\text{MAD}] = \frac{1}{9}\sqrt{\frac{2}{\pi n}} \sum_{d=1}^{9} \sqrt{p_d(1 - p_d)} \;\approx\; \frac{0.235}{\sqrt{n}}$$

Table 2: Copula properties and recommended accounting applications.

Copula	Lower Tail	Upper Tail	Use Case
Gaussian	None	None	General correlation
Clayton	Yes	None	Payment defaults, distress
Gumbel	None	Yes	High-value co-occurrence
Frank	None	None	Symmetric, no extremes
Student-t	Yes	Yes	Joint tail events

where the numerical constant follows from evaluating the sum over Benford probabilities. This ensures convergence to "close conformity" (MAD < 0.006*) [23] for* $n \gtrsim 1{,}600$*.*

5.2 Copula-Based Dependency Modeling

Real accounting data exhibits complex dependencies: transaction amounts correlate with line-item counts, approval levels depend on amounts, and payment timing relates to invoice size. DataSynth separates marginal distributions from dependency structure using copulas [22].

Definition 5.2 (Copula)**.** *A* d*-dimensional copula* $C : [0,1]^d \to [0,1]$ *is a joint CDF with uniform marginals. By Sklar's theorem, any joint distribution* H *with marginals* $F_1, \ldots, F_d$ *can be written as* $H(x_1, \ldots, x_d) = C(F_1(x_1), \ldots, F_d(x_d))$*.*

DataSynth implements five copula families:

Sampling algorithms: Gaussian via Cholesky decomposition, Clayton via conditional method, Gumbel via Marshall-Olkin with positive stable distribution, Frank via conditional inversion, Student-t via chi-squared mixing (see Appendix C).

5.3 Mixture Distributions for Transaction Amounts

Enterprise transaction amounts cluster into distinct regimes (routine, significant, major). DataSynth models this via K-component log-normal mixtures:

$$f(x) = \sum_{k=1}^{K} w_k \cdot \frac{1}{x\sigma_k\sqrt{2\pi}} \exp\left(-\frac{(\ln x - \mu_k)^2}{2\sigma_k^2}\right), \quad \sum_{k=1}^{K} w_k = 1. \tag{9}$$

The default three-component configuration mimics empirical distributions observed in ERP data:

Component	w_k	μ_k	σ_k	Mean ($e^{\mu+\sigma^2/2}$)
Routine	0.60	6.0	1.5	\$1,245
Significant	0.30	8.5	1.0	\$8,103
Major	0.10	11.0	0.8	\$82,269

Additional distribution families include Gaussian mixtures (for real-valued domains such as FX adjustments), Pareto (heavy tails), Weibull (time-to-event), Beta (proportions), and zero-inflated models.

5.4 Temporal Dynamics

Financial data exhibits rich temporal structure that forms part of the statistical knowledge layer.

Business-day calendars. DataSynth implements holiday calendars for 15 regions (US, DE, GB, FR, IT, ES, CA, CN, JP, IN, BR, MX, AU, SG, KR) with configurable half-day policies, month-end conventions (modified following, preceding, end-of-month), and T+N settlement rules for equities (T+2), government bonds (T+1), and FX spot (T+2).

Period-end dynamics. Transaction volumes spike near period ends. We model this with configurable decay curves:

$$\lambda(t) = \lambda_0 \cdot \begin{cases} 1 + (m_{\text{peak}} - 1) \cdot e^{-\alpha(T-t)} & \text{(exponential)} \\ 1 + (m_{\text{peak}} - 1) \cdot (1 - t/T)^{-\beta} & \text{(extended crunch)} \end{cases} \tag{10}$$

where T is the period-end date, m_{peak} is the peak multiplier (typically 3.5× for month-end, 6.0× for year-end), and α, β are decay parameters. In the extended-crunch model the intensity diverges as $t \to T$; in practice, DataSynth clamps $\lambda(t)$ at $\lambda_0 \cdot m_{\text{peak}}$ and truncates the active window to $[T + \text{start\_day},\ T]$.

Processing lags. Real posting dates lag event dates. DataSynth models this with log-normal lag distributions per event type and a cross-day posting model where the probability of next-day posting increases with hour of origination.

Intraday patterns. Within each business day, transaction intensity follows configurable segments: morning spike (1.8× at 08:30–10:00), lunch dip (0.4× at 12:00–13:30), and end-of-day rush (1.5× at 16:00–17:30).

Economic cycles and drift. Long-horizon datasets incorporate regime changes via:

$$\mu(t) = \mu_0 \cdot \left(1 + A \sin\left(\frac{2\pi t}{P}\right)\right) \cdot \mathbb{1}_{\text{expansion}} + \mu_0(1 - d_r) \cdot \mathbb{1}_{\text{recession}} \tag{11}$$

where P is the cycle period (default 48 months), A is the amplitude (0.15), and d_r is the recession depth (0.25).

5.5 Causal Transfer Functions

The counterfactual simulation engine (Section 10) models inter-process dependencies via a causal DAG whose edges carry eight typed transfer functions: Linear, Exponential, Logistic, Inverse Logistic, Step, Threshold, Decay, and Piecewise. All transfer functions compose with an edge weight $w_e \in [0, 1]$ and are clamped to target node domain bounds. Full details are in Section 10.

6 Generation Pipeline

DataSynth's generation pipeline transforms a declarative YAML configuration into a complete, internally consistent enterprise dataset—producing the *structural knowledge layer* ($\mathcal{K}_S$) of the reference knowledge graph.

6.1 Orchestration Workflow

The `GenerationOrchestrator` executes the following stages:

1. **Configuration loading and validation.** Parse YAML, apply industry preset defaults, validate invariants.
2. **Chart of Accounts initialization.** Generate a GL account structure at the configured complexity level (small: ∼100, medium: ∼400, large: ∼2,500 accounts).
3. **Master data generation.** Produce vendors, customers, materials, fixed assets, employees, and cost centers with cross-referenced relationships.
4. **Opening balance generation.** Establish period-start balances satisfying the accounting equation $A = L + E$.
5. **Document-flow generation.** Execute P2P, O2C, S2C, HR, manufacturing, tax, treasury, ESG, and project accounting generators.
6. **Period-close processing.** Run accruals, depreciation, FX translation, intercompany elimination, and year-end closing. Generate trial balances and financial statements.
7. **Standards generation.** Produce accounting standards artifacts (ASC 606 contracts, ASC 842 leases, fair value measurements, impairment tests) and audit standards artifacts (ISA mappings, confirmations, audit opinions, SOX assessments).
8. **Audit methodology execution.** When FSM-driven audit generation is enabled, the orchestrator loads the configured methodology blueprint and generation overlay, constructs an engagement context from the generated financial data, and runs the finite state machine engine. The engine walks procedures in precondition DAG order, executing each procedure's state machine and dispatching step commands to the appropriate generators. Streaming execution is supported: a callback-based API emits each audit event as it is produced, enabling real-time downstream consumption without buffering the full engagement result. The resulting event trail and artifact bag are integrated into the standard output pipeline.
9. **Anomaly injection.** Apply configured anomalies with ground-truth labels across all five knowledge dimensions.
10. **Data quality variation.** Optionally inject missing values, format variations, typos, and encoding issues.
11. **Evaluation and graph export.** Run coherence validators, construct the reference knowledge graph, and export to configured formats.

12. **Output.** Stream results to configured sinks (CSV, JSON, Parquet).

6.2 Journal Entry Generation

Journal entries are the atomic unit of accounting data.

Definition 6.1 (Balanced Entry)**.** *A journal entry j with line items $\{l_1, \ldots, l_n\}$ is* balanced *iff:*

$$\sum_{i:l_i.debit} l_i.amount = \sum_{i:\neg l_i.debit} l_i.amount. \tag{12}$$

DataSynth enforces Equation (12) at construction time: the `JournalEntry` constructor returns an error if the debit-credit difference exceeds a configurable tolerance (default: \$0.01). This invariant is part of the *normative knowledge layer* and is maintained across all generators.

Line-item amounts are drawn from the configured mixture distribution (Equation (9)). The number of line items per entry follows a correlated draw from the copula model, ensuring that higher-amount entries tend to have more line items and require higher approval levels.

6.3 Document Flow Generation

Enterprise transactions span multiple documents with strict sequencing and referential integrity.

Procure-to-Pay (P2P).

$$\text{Purchase Order} \rightarrow \text{Goods Receipt} \rightarrow \text{Vendor Invoice} \rightarrow \text{Payment}$$

Each stage creates the corresponding document record, updates the document-chain manager with forward/backward references, and posts journal entries to the appropriate GL accounts.

Three-Way Matching. The three-way matcher validates consistency across PO, GR, and invoice with quantity tolerance ($\epsilon_q = 2\%$), price tolerance ($\epsilon_p = 5\%$), and over-delivery allowance ($\delta_{\max} = 10\%$).

Order-to-Cash (O2C).

$$\text{Sales Order} \rightarrow \text{Delivery} \rightarrow \text{Customer Invoice} \rightarrow \text{Customer Receipt}$$

with revenue recognition, AR postings, and cash receipts. Partial deliveries and split invoicing are supported.

Source-to-Contract (S2C). Full sourcing lifecycle: sourcing projects, supplier qualifications, RFx events, supplier bids, bid evaluations, procurement contracts, catalog items, and supplier scorecards.

HR/Payroll. Payroll runs with line items (base pay, overtime, deductions, taxes), time entries, expense reports, and benefit enrollment (health, dental, vision, retirement) with employer/employee contributions.

Manufacturing. Production orders, routing operations, quality inspections with characteristics, cycle counts, multi-level BOMs with phantom assembly support, and inventory movement tracking (receipt, issue, transfer, return, scrap, adjustment). As of v2.2, the manufacturing pipeline implements full standard cost accounting with WIP-to-FG-to-COGS flow:

$$\text{FG}_{\text{close}} = \text{FG}_{\text{open}} + \text{completions} - \text{COGS} - \text{scrap}$$

Material price variances, labor rate variances, and overhead volume variances are computed against standard costs and posted to designated variance accounts. Quality inspection failure rates feed a warranty provision generator that creates provisions per IAS 37/ASC 450, ensuring that manufacturing quality data flows through to the financial statements.

6.4 Tax Accounting

DataSynth generates comprehensive tax accounting data spanning multiple jurisdictions:

- **Tax jurisdictions and codes:** Multi-level jurisdiction hierarchies (federal, state, local) with associated tax codes, rates, and effective dates.
- **Tax lines and returns:** Per-transaction tax calculations, periodic tax return preparation with aggregated liabilities.
- **Tax provisions:** Current and deferred tax calculations under ASC 740/IAS 12, including temporary differences, deferred tax rollforward, and rate reconciliation. As of v2.2, tax provisions derive from actual GL pre-tax income rather than independent estimates, ensuring that the effective tax rate reconciliation ties to the income statement.
- **Transaction-level VAT/GST:** Tax postings are generated from source documents (invoices, receipts), producing input and output VAT lines that reconcile to periodic tax returns.
- **Withholding tax:** Source-based withholding records with jurisdiction-specific rates and treaty provisions.
- **Uncertain tax positions:** UTP assessment with probability weighting and disclosure requirements.
- **Transfer pricing:** Intercompany pricing methods (comparable uncontrolled, resale, cost-plus, TNMM, profit split) with arm's-length validation.

6.5 Treasury Management

The treasury module generates:

- **Cash positioning:** Daily cash positions across accounts and entities with automated sweeping via cash pools.
- **Cash forecasting:** Rolling forecasts with multiple time horizons and confidence intervals.

- **Hedging instruments:** Forward contracts, options, and swaps with hedge relationship documentation and effectiveness testing under ASC 815/IFRS 9. In v2.2, hedging instruments generate mark-to-market journal entries routed according to hedge designation: cash flow hedges to OCI, fair value hedges to P&L, with reclassification entries upon hedge settlement or ineffectiveness.
- **Debt instruments:** Term loans, revolving facilities, and bonds with amortization schedules and covenant tracking. Debt instruments now generate periodic interest accrual journal entries that tie to the interest expense line in the income statement.
- **Debt covenants:** Financial covenant calculations with compliance monitoring and breach detection. Covenant ratios (e.g., debt-to-EBITDA, interest coverage) are evaluated against actual financial ratios derived from the generated GL data.

6.6 Financial Statement Package

Version 2.2 introduces a comprehensive financial statement package that transforms raw GL data into publication-ready financial reports:

- **Cash flow enhancer:** The indirect-method cash flow statement is derived from actual balance sheet movements and income statement data, with non-cash adjustments (depreciation, amortization, unrealized FX gains/losses) automatically identified and reconciled.
- **Segment reporting:** Operating segments are constructed from real journal entry data using configurable allocation rules. Segment revenue, operating profit, and assets reconcile to the consolidated totals, with inter-segment eliminations tracked separately.
- **Dividend generation:** Dividend declarations, accruals, and payments are generated based on configurable payout ratios applied to retained earnings, with journal entries flowing through to the statement of changes in equity.
- **XBRL export:** Financial statements are exported as XBRL 2.1 instance documents with US GAAP or IFRS taxonomies, enabling automated compliance testing and regulatory submission workflows.

6.7 ESG and Sustainability

The ESG module generates sustainability data aligned with major reporting frameworks:

- **Environmental:** GHG emissions (Scope 1/2/3), energy consumption by source, water usage, waste records by type and disposition, and climate scenario analysis.
- **Social:** Workforce diversity metrics, pay equity analysis, safety incidents and metrics, materiality assessments.
- **Governance:** Board composition metrics, supplier ESG assessments, ESG disclosure documents.

6.8 Project Accounting

Project-based organizations require WBS-structured cost tracking:

- **Projects:** Multi-level project structures with budget allocation and resource planning.
- **Cost lines:** Labor, material, and overhead cost postings with project/WBS element attribution.
- **Revenue recognition:** Percentage-of-completion and milestone-based revenue for long-term contracts.
- **Earned value:** BCWS, BCWP, ACWP calculations with derived metrics (CPI, SPI, EAC, ETC).
- **Change orders:** Scope and budget modifications with approval tracking.
- **Milestones:** Deliverable tracking with planned vs. actual completion dates.

6.9 Subledger and Period-Close Processing

Subledger generation. Four subledger generators maintain detailed subsidiary records: AR (customer invoices, receipts, aging, credit memos), AP (vendor invoices, payments, aging, prepayments), FA (capitalization, depreciation, disposals, impairment), and Inventory (receipts, issues, valuation methods). Each reconciles to its control account in the GL.

Period-close engine. Executes: (i) accrual entries, (ii) depreciation postings, (iii) FX translation with CTA postings, (iv) intercompany matching and elimination, (v) year-end income-to-equity reclassification, and (vi) financial statement generation (balance sheet, income statement, cash flow, changes in equity).

6.10 Intercompany Processing

Multi-entity datasets require IC transaction matching and elimination for consolidated reporting. The IC generator creates bilateral transaction pairs. The matching engine identifies matched pairs within configurable tolerances. The elimination generator produces consolidation entries that net IC receivables/payables, IC revenue/expense, and unrealized profit in inventory.

6.11 Accounting and Audit Standards

The normative knowledge layer ($\mathcal{K}_N$) is generated through:

Accounting standards. Revenue recognition (ASC 606/IFRS 15) with customer contracts and performance obligations. Leases (ASC 842/IFRS 16) with ROU assets and lease liabilities. Fair value measurements (ASC 820/IFRS 13) with Level 1/2/3 hierarchy. Impairment testing (ASC 360/IAS 36). ECL models and provision matrices. Business combinations with purchase price allocation. Pension obligations and stock compensation.

Audit standards. ISA compliance (34 standards) with procedure mapping. PCAOB standards (19+) with cross-reference. Analytical procedures (ISA 520), confirmations (ISA 505), audit opinions (ISA 700/705/706/701), key audit matters. Group audit (ISA 600) with component auditor model.

Internal controls. COSO 2013 framework with 5 components, 17 principles, and maturity levels. 12 transaction-level controls (C001–C060) and 6 entity-level controls (C070–C081). Segregation of duties enforcement. SOX 302/404 assessments with deficiency classification and material weakness identification.

Regulatory exports. FEC (French *Fichier des Écritures Comptables*) and GoBD (German) audit-ready exports for jurisdiction-specific tax authority requirements.

7 Reference Knowledge Graph Construction

This section describes how DataSynth constructs reference knowledge graphs from the generated enterprise data, encoding all three knowledge layers (Definition 2.13) into graph structures suitable for GNN-based audit analytics.

7.1 Graph-Theoretic Foundation

Double-entry bookkeeping has a natural directed-graph representation [13]: each distinct GL account forms a node, credit entries produce outgoing edges, and debit entries produce incoming edges. Following Ivertowski et al. [17], we adopt a temporal formulation:

$$G(t_0, t_1) = \big(A(t_0, t_1),\ E(t_0, t_1)\big) \tag{13}$$

where $A(t_0, t_1)$ is the set of account nodes with monetary state over $[t_0, t_1]$, and each edge $E_i(t_0, t_1) = \sum_{n=1}^{|\Omega_{A_i}|} \omega_n$ aggregates all transactions ω_n affecting node A_i in that period. This temporal definition enables system-theoretic analyses such as cross-correlation and auto-correlation on account flows.

Complementing accounting networks constructed from real data, DataSynth's graphs are generated *forward*: every edge has known provenance, eliminating the combinatorial matching ambiguity of Proposition 2.3.

7.2 Entity Type Coverage

The reference knowledge graph encompasses 78+ entity types across the full enterprise data stack, organized by domain:

7.3 Relationship Edge Types

Entities are connected by 39+ typed relationship edges with explicit cardinality constraints:

- **Transactional edges:** Debit/credit flows between accounts (one-to-many), document chain references (PO→GR→Invoice→Payment), payment allocations.

Table 3: Entity type coverage in the reference knowledge graph, organized by enterprise domain.

Domain	Types	Representative Entities
Accounting	6	JournalEntry, ChartOfAccounts, ACDOCA, AccountBalance, TrialBalance, FiscalPeriod
Master Data	6	Vendor, Customer, Material, FixedAsset, Employee, EntityRegistry
Document Flow	10	PurchaseOrder, GoodsReceipt, VendorInvoice, Payment, SalesOrder, Delivery, CustomerInvoice, CustomerReceipt
Sourcing (S2C)	9	SourcingProject, RfxEvent, SupplierBid, ProcurementContract, CatalogItem, SupplierScorecard
Financial Reporting	6	FinancialStatement, ManagementKpi, Budget, ConsolidationSchedule, OperatingSegment
HR/Payroll	6	PayrollRun, TimeEntry, ExpenseReport, BenefitEnrollment, DefinedBenefitPlan, StockGrant
Manufacturing	6	ProductionOrder, RoutingOperation, QualityInspection, CycleCount, BomComponent
Tax	8	TaxJurisdiction, TaxCode, TaxReturn, TaxProvision, DeferredTaxRollforward, UncertainTaxPosition
Treasury	6	CashPosition, CashForecast, CashPool, HedgingInstrument, DebtInstrument, DebtCovenant
ESG	8	EmissionRecord, EnergyConsumption, WaterUsage, WasteRecord, WorkforceDiversityMetric, SafetyIncident
Project	5	Project, ProjectCostLine, EarnedValueMetric, ChangeOrder, ProjectMilestone
Audit	8	EngagementLetter, AuditOpinion, KeyAuditMatter, ComponentAuditor, GroupAuditPlan, SamplingPlan
Controls	4	InternalControl, ControlMapping, SoD, CosoComponent

- **Hierarchical edges:** Chart of accounts parent/child, cost center hierarchy, organizational reporting lines, corporate group structure.
- **Master data edges:** Vendor-to-material linkages, customer-to-segment assignments, employee-to-department.
- **Control edges:** Approval relationships with threshold attributes, segregation-of-duties constraint edges, COSO principle-to-control mappings.
- **Standards edges:** GAAP rule applicability, ISA requirement mappings, SOX assessment linkages.
- **Audit trail edges:** When the FSM-driven audit engine is active, additional edges connect engagement nodes to workpapers, workpapers to evidence, findings to assessed risks, and audit opinions to findings—each annotated with the governing ISA or IIA-GIAS procedure step that produced the artifact.
- **Cross-process edges:** Inventory P2P-O2C links (GoodsReceipt → Delivery), payment-bank reconciliation, intercompany bilateral matching.

Each edge carries a type discriminator, cardinality constraint (one-to-one, one-to-many, many-to-many), and domain-specific attributes (amounts, dates, status flags).

7.4 Three-Layer Graph Encoding

The reference knowledge graph explicitly encodes all three knowledge layers as graph properties:

Layer 1 (Structural). Node existence, edge connectivity, document chain integrity, master data relationships. The graph builder constructs three complementary subgraphs:

- **Transaction graph:** Nodes are accounts/entities; edges are transactions with amount, date, and process-type features.
- **Approval network:** Nodes are users; edges are approval relationships with threshold attributes. SoD conflicts are detectable as structural motifs.
- **Entity relationship graph:** Nodes are legal entities, vendors, customers; edges represent ownership, control, and supply-chain relationships.

Layer 2 (Statistical). Node and edge properties carry distributional metadata: the `ToNodeProperties` trait maps domain models to seven property-value types (`String`, `Decimal`, `Integer`, `Boolean`, `Date`, `DateTime`, `List`), ensuring lossless round-tripping through graph serialization. Amount distributions, temporal patterns, and correlation structures are recoverable from the graph properties.

Layer 3 (Normative). Control nodes encode COSO component/principle mappings, maturity levels, and scope assignments. Standards nodes encode GAAP/IFRS rule applicability and audit procedure requirements. Anomaly labels are attached as edge/node attributes with full provenance (type, category, difficulty, severity, scheme membership).

7.5 Hypergraph Extensions

Some enterprise relationships are inherently multi-entity. A hypergraph builder extends the model to capture these:

- **Three-way match hyperedge:** Links a purchase order, goods receipt, and vendor invoice simultaneously.
- **Intercompany elimination:** Links the matched pair of IC transactions with their consolidation entry.
- **Period-close:** Links accrual entries, depreciation postings, and the resulting financial statement line items.

7.6 Export Formats

The reference knowledge graph is exported in three formats optimized for different downstream use cases:

- **PyTorch Geometric (`.pt`):** Heterogeneous graph tensors with typed nodes and edges, directly loadable for GNN training. Node features are scaled and normalized; edge indices use COO format.
- **Neo4j (CSV + Cypher):** Node and relationship CSV files with Cypher import scripts for graph database analysis, Cypher queries, and visualization.
- **DGL (heterograph):** Typed heterogeneous graph with node/edge feature dictionaries for Deep Graph Library research.

7.7 Process Mining (OCEL 2.0)

The OCPM module generates Object-Centric Event Logs conforming to the OCEL 2.0 standard [9]. Events are linked to multiple typed objects via many-to-many relationships. Lifecycle state machines for `PurchaseOrder`, `SalesOrder`, and `VendorInvoice` objects define probabilistic transitions, producing realistic variant distributions.

Multi-object correlation events (three-way match, payment allocation, bank reconciliation) link objects across process boundaries. Resource pool workload modeling assigns events to users via configurable strategies (round-robin, least-busy, skill-based). Enriched events carry state, workload, and correlation-ID attributes for conformance checking and resource-perspective analysis.

The OCEL 2.0 logs are fully traceable to the reference knowledge graph—every event corresponds to a known structural, statistical, and normative context, enabling process mining research with ground truth.

7.8 Ground Truth Properties of the Reference Graph

By construction, the reference knowledge graph satisfies the completeness property of Proposition 2.14. Concretely, for any query over the graph:

1. **Node classification:** Every entity's type, attributes, and anomaly status are known. This provides ground-truth labels for node classification tasks in GNN research.
2. **Edge prediction:** Every relationship is generated with known provenance. Missing edges in a subgraph can be identified against the full graph, enabling link prediction benchmarks.
3. **Anomaly detection:** Every injected anomaly is labeled with type, difficulty, severity, and causal chain. Detection algorithms can be evaluated against this ground truth across all five knowledge dimensions (Definition 2.10).
4. **Community structure:** Vendor clusters, customer segments, intercompany groups, and departmental boundaries are known, enabling evaluation of community detection algorithms.

Table 4: Anomaly taxonomy with representative types per category.

Category	Count	Representative Types
Fraud	54	FictitiousEntry, RoundDollarManipulation, JustBelowThreshold, SelfApproval, DuplicatePayment, FictitiousVendor, RevenueManipulation
Error	25	DuplicateEntry, ReversedAmount, WrongPeriod, WrongAccount, MissingReference, TransposedDigits, CutoffError
Process	21	LatePosting, SkippedApproval, ThresholdManipulation, MissingDocumentation, AfterHoursPosting
Statistical	17	UnusualAmount, TrendBreak, BenfordViolation, UnusualFrequency, SeasonalAnomaly
Relational	14	CircularTransaction, DormantAccountActivity, UnusualAccountPair, NewCounterparty, CentralityAnomaly

5. **Temporal evolution:** The graph can be sliced by period, enabling temporal GNN research on time-evolving audit knowledge.

8 Anomaly Injection Framework

A primary use case for DataSynth is generating labeled datasets for fraud detection and anomaly analytics. Anomalies are injected with full provenance across all five knowledge dimensions (Definition 2.10), providing comprehensive ground-truth labels that are difficult to obtain from real enterprise data alone.

8.1 Anomaly Taxonomy

DataSynth defines anomaly types organized into five categories, with over 130 fine-grained subtypes in the codebase. Table 4 summarizes the primary types used in the default injection framework.

Each anomaly type maps to specific knowledge dimensions: fraud types span structural (fake entities), statistical (unusual amounts), and normative (control bypasses) dimensions; relational types are inherently topological; error types span statistical and temporal dimensions. This multi-dimensional labeling enables evaluation of detection methods across all five dimensions (Definition 2.10).

8.2 Multi-Stage Fraud Schemes

Real fraud often evolves through stages over extended time horizons. DataSynth implements three multi-stage *fraud schemes*:

1. Vendor Kickback Scheme. A buyer routes payments to a shell company they control. Stages: (a) establish shell vendor, (b) route initial small invoices, (c) gradually increase amounts, (d) introduce collusion patterns.

2. Gradual Embezzlement Scheme. Small, individually undetectable amounts siphoned over time. Stages: (a) initial testing with minimal amounts, (b) establishment of regular extraction, (c) acceleration in amount and frequency, (d) period-end concentration.

3. Revenue Manipulation Scheme. Fictitious customers and premature revenue recognition. Stages: (a) customer creation, (b) fictitious order entry, (c) premature recognition, (d) channel stuffing with period-end concentration.

The `SchemeAdvancer` tracks each scheme's state via probability-weighted state machines. Each transaction within a scheme receives a `MultiStageAnomalyLabel` linking it to the scheme instance, stage, and causal chain.

8.3 Anomaly Correlation Patterns

Anomalies in real data are not independently distributed. DataSynth models three correlation mechanisms:

Temporal Clustering. Anomalies concentrate in time windows with configurable intensity multipliers:

$$P(\text{anomaly at } t) = p_0 \cdot \prod_{w \in \mathcal{W}} (1 + (m_w - 1) \cdot \mathbb{1}_{t \in w}) \tag{14}$$

Cascade Propagation. One anomaly triggers follow-on anomalies in dependent processes via `CascadeStep` definitions with probability and delay parameters.

Co-Occurrence. The `CoOccurrencePattern` defines allowed combinations and prevents unrealistic pairings.

8.4 Difficulty, Severity, and Confidence Scoring

Each injected anomaly receives three scores:

Difficulty. A composite score $d \in [0, 1]$ from five factor groups:

$$d = \sum_{f \in \mathcal{F}} w_f \cdot s_f, \quad \mathcal{F} = \{\text{amount}, \text{blending}, \text{collusion}, \text{concealment}, \text{temporal}\} \tag{15}$$

where $w_f \geq 0$, $\sum_f w_f = 1$, and each factor score $s_f \in [0, 1]$.

Severity. A five-component organizational impact model:

$$\text{severity} = 0.25 \cdot s_{\text{type}} + 0.30 \cdot s_{\text{monetary}} + 0.20 \cdot s_{\text{frequency}} + 0.15 \cdot s_{\text{scope}} + 0.10 \cdot s_{\text{timing}} \tag{16}$$

Confidence. Detection confidence combining pattern-match, rule-violation, and behavioral-deviation scores with contextual modifiers.

8.5 Ground-Truth Labels

Every anomaly produces a structured label record containing: anomaly type/category/subcategory, affected transaction IDs and account codes, difficulty/severity/confidence scores, scheme identifier and stage, contributing factors with impact scores, and temporal context. Labels are exported as separate CSV/JSON files, enabling supervised, semi-supervised, and unsupervised evaluation protocols.

Because the labels are generated by the same forward process that produces the data (Proposition 2.14), they provide complete and consistent ground truth with respect to the generative model—complementing labels derived from real data, which, while grounded in actual patterns, are typically incomplete and may be influenced by the same systematic errors they seek to identify.

8.6 Banking-Specific Anomalies: AML Typologies

The banking module extends the framework with five AML typologies: (1) Structuring/smurfing, (2) Funnel accounts, (3) Layering across jurisdictions, (4) Mule networks, and (5) Round tripping with 30–60 day delays. Each generates persona-based patterns with risk-tier profiles and sophistication levels.

8.7 Fraud Scenario Packs

Five pre-built fraud scenario packs—revenue fraud, payroll ghost, vendor kickback, management override, and comprehensive—provide complete anomaly configurations that can be deep-merged into any generation config.

9 Privacy-Preserving Fingerprint Module

While DataSynth can generate reference knowledge graphs from scratch using configured distributions, a key scenario involves calibrating the generative model to match real enterprise data. The fingerprint module enables this by extracting *distributional summaries* under formal privacy guarantees, bridging the gap between the forward generation paradigm (Section 2.5) and the statistical characteristics of production data.

9.1 Privacy Mechanisms

The module combines two complementary techniques:

Differential Privacy (ε-DP). Each extracted statistic is protected by calibrated Laplace noise:

$$\tilde{s} = s + \text{Lap}\left(\frac{\Delta s}{\varepsilon}\right) \tag{17}$$

where s is the true statistic, Δs is its sensitivity, and ε is the privacy budget. A privacy accountant tracks cumulative ε expenditure.

Four privacy levels provide pre-configured ε values:

Level	ε	k-anon	Privacy	Utility
Minimal	5.0	3	Low	High
Standard	1.0	5	Medium	Medium
High	0.5	10	High	Reduced
Maximum	0.1	20	Maximum	Low

k-Anonymity. Categorical distributions with fewer than k occurrences are suppressed.

9.2 Fingerprint Extraction and Synthesis

The `FingerprintExtractor` processes input data in a streaming fashion through seven components: schema extraction, statistics extraction, correlation extraction, integrity extraction, rule extraction, anomaly extraction, and privacy audit. Each numeric statistic is noised via Equation (17).

The fingerprint is stored as a ZIP archive (`.dsf`) containing manifest, schema, statistics, correlations, integrity, rules, anomalies, and privacy audit components.

The `ConfigSynthesizer` reads a `.dsf` fingerprint and generates a complete DataSynth YAML configuration through scale estimation, distribution mapping, copula reconstruction, anomaly calibration, and seed assignment. This two-phase architecture (extract $\rightarrow$ synthesize) cleanly separates the *privacy boundary* from the *generation process*.

9.3 Privacy-Utility Tradeoff

Proposition 9.1 (Utility bound)**.** *For n records with true mean μ and column range R: $\mathbb{E}[|\tilde{\mu} - \mu|] = R/(n\varepsilon)$. For $n = 10{,}000$, $R = 10^6$, $\varepsilon = 1.0$: $\mathbb{E}[|\tilde{\mu} - \mu|] = 100$, which is $< 0.01\%$ of the range—negligible for distribution fitting.*

This demonstrates that for enterprise-scale datasets ($n > 10{,}000$), even standard privacy levels introduce negligible utility loss, making the fingerprint approach practical for calibrating reference knowledge graphs against real-world statistical properties while maintaining formal privacy guarantees.

9.4 Workflow

1. **Extract:** `datasynth-data fingerprint extract --input data.csv \`
   `--output fp.dsf --privacy-level standard`
2. **Validate:** `datasynth-data fingerprint validate fp.dsf`
3. **Generate:** `datasynth-data generate --fingerprint fp.dsf --output ./synthetic/`
4. **Evaluate:** `datasynth-data fingerprint evaluate --fingerprint fp.dsf \`
   `--synthetic ./synthetic/`

The fingerprint file can be safely shared across organizational boundaries, enabling generation of calibrated reference knowledge graphs without exposing individual records.

10 Counterfactual Simulation

A growing need in audit analytics is the ability to answer "what if" questions: *What would the reference knowledge graph look like under a recession? How would a control failure propagate through the enterprise?* DataSynth's counterfactual simulation engine models causal dependencies between financial processes, applies structured interventions, and generates paired baseline/counterfactual reference knowledge graphs suitable for causal inference research.

10.1 Causal DAG Model

Definition 10.1 (Financial Causal DAG)**.** *A* financial causal DAG *is a tuple* $\mathcal{C} = (V, E, \tau, w)$ *where:*

- $V = \{v_1, \ldots, v_n\}$ *is a set of process nodes with baseline values, bound intervals, and categories (Macro, Operational, Control, Financial, Behavioral, Regulatory, Outcome).*
- $E \subseteq V \times V$ *is a set of directed acyclic edges.*
- $\tau : E \to \mathcal{T}$ *assigns transfer functions from a family of eight types (Linear, Exponential, Logistic, Inverse Logistic, Step, Threshold, Decay, Piecewise).*
- $w : E \to [0, 1]$ *assigns strength weights.*

Each edge carries a lag $\lambda_e \in \mathbb{N}_0$ *(months) encoding temporal delay.*

The default DAG contains 17 nodes spanning macroeconomic drivers (GDP growth, interest rates), operational parameters (transaction volume, staffing, automation), control variables, financial metrics, and regulatory factors.

Causal propagation. Given interventions $\mathcal{I} = \{(v_j, \hat{x}_j)\}$, the engine computes equilibrium via forward propagation in topological order (Algorithm 1).

10.2 Intervention Framework

Eight intervention types are supported: ParameterShift (with linear, exponential, logistic, and step interpolation), ControlFailure (effectiveness reduction, bypass, intermittent, delayed detection), MacroShock (recession, inflation, currency crisis, pandemic, etc.), EntityEvent (vendor default, customer churn, M&A, etc.), ProcessChange (threshold changes, automation, reorganization), RegulatoryChange (new standards, materiality changes), Composite (bundled with conflict resolution), and Custom.

Each intervention specifies timing (start month, duration) and onset type: sudden, gradual ($\alpha(t) = \min((t - t_{\text{start}})/r, 1)$), or oscillating ($\alpha(t) = \frac{1}{2}(1 - \cos(\pi(t - t_{\text{start}})/r))$).

10.3 Paired Generation and Diff Analysis

The `ScenarioEngine` produces paired datasets: a baseline and a counterfactual, both sharing the same deterministic seed. Differences are attributable solely to the intervention. The `DiffEngine` produces four views: ImpactSummary (KPI deltas), RecordLevelDiff (per-file

Algorithm 1: Causal forward propagation with lag awareness.

Input : Causal DAG $\mathcal{C} = (V, E, \tau, w)$; interventions $\mathcal{I} = \{(v_j, \hat{x}_j)\}$; current month t
Output : Node values $\{x_i\}_{i=1}^{|V|}$

Compute topological order σ via Kahn's algorithm;
foreach $v_i \in V$ **do**
  $x_i \leftarrow b_i$; `// initialize to baseline`
foreach $(v_j, \hat{x}_j) \in \mathcal{I}$ **do**
  $x_j \leftarrow \hat{x}_j$; `// apply direct interventions`
for $k = 1$ **to** $|V|$ **do**
  $v_i \leftarrow \sigma_k$;
  if $(v_i, \cdot) \in \mathcal{I}$ **then**
    continue;
  $\Delta_i \leftarrow 0$;
  foreach *edge* $e = (v_p, v_i) \in E$ **do**
    if $t \geq \lambda_e$ **then**
      $\delta_p \leftarrow x_p - b_p$;
      $\Delta_i \leftarrow \Delta_i + \tau_e(\delta_p) \cdot w_e$;
  $x_i \leftarrow \text{clamp}(b_i + \Delta_i,\ \ell_i,\ u_i)$;
return $\{x_i\}_{i=1}^{|V|}$;

changes), AggregateComparison (distribution shifts), and InterventionTrace (causal attribution through the DAG).

10.4 Scenario Constraints

Four built-in constraints ensure counterfactual datasets remain valid reference knowledge graphs: accounting identity preservation, document chain integrity, period close execution, and balance coherence ($A = L + E$). Users may define additional constraints as range bounds on config paths.

11 Evaluation Framework

DataSynth includes a built-in evaluation suite that validates the statistical, structural, and domain-specific quality of generated data across all three knowledge layers.

11.1 Statistical Validation

Benford's Law Tests. First-digit and first-two-digit distributions are tested via MAD, chi-squared, and Kolmogorov-Smirnov statistics against theoretical Benford probabilities [23].

Distribution Fitting. Anderson-Darling tests for log-normal fit at configurable significance levels (default $\alpha = 0.05$).

Correlation Verification. Pearson correlation matrix comparison against configured targets.

Temporal Pattern Analysis. Intraday profiles, day-of-week seasonality, month-end concentration, CUSUM and EWMA control charts for drift detection.

11.2 Coherence Validators

DataSynth implements 25+ domain-specific coherence validators:

Table 5: Coherence validators and the invariants they enforce.

Validator	Key Invariants
Balance Sheet	Assets = Liabilities + Equity; period-over-period consistency
Document Chain	PO→GR→Invoice→Payment referential integrity
Subledger Reconciliation	AR/AP/FA/Inventory subledger totals = GL control accounts
Intercompany Elimination	IC receivables = IC payables; elimination nets to zero
HR / Payroll	Gross = Net + Deductions; tax withholding bounds
Manufacturing	BOM costing ≤ production cost; scrap accounting
Sales Quotes	Quote→order conversion; discount adherence
Sourcing	Bid evaluation completeness; scorecard ranges
Bank Reconciliation	Book balance + outstanding = bank balance
Financial Reporting	IS → BS via retained earnings; BS → CF via working capital
Standards	ASC 606 obligation satisfaction; ASC 842 ROU amortization
Network Analysis	Vendor concentration; customer segment revenue shares
Tax	Tax rate application; withholding accuracy
Treasury	Cash flow consistency; covenant compliance
Project Accounting	Budget vs. actual variance within thresholds
v2.2 capstone validators	
COGS / WIP Reconciliation	$\text{FG}_{\text{close}} = \text{FG}_{\text{open}} + \text{completions} - \text{COGS} - \text{scrap}$; WIP rollforward; variance reconciliation
IC Elimination Completeness	All IC transaction pairs have corresponding elimination entries
Interest Expense Proof	GL interest expense = $\sum$ accrued interest from debt instruments
ETR Reconciliation	Effective tax rate = statutory rate ± permanent differences
Hedge Effectiveness	Hedge ratio within 80–125% corridor (ASC 815/IFRS 9)
Cash Flow Reconciliation	Net cash = opening cash + operating + investing + financing
Equity Rollforward	Closing equity = opening + net income − dividends ± OCI ± other
Segment Reconciliation	$\sum$ segment totals + eliminations = consolidated totals
Trial Balance Master Proof	$\sum$ all JEs from all modules = closing trial balance

11.3 ML-Readiness Assessment

For datasets intended for machine learning, DataSynth evaluates feature quality (Wasserstein distance, cardinality, missing-data patterns), label quality (distribution, correlation analysis, completeness), split metrics (temporal train/val/test balance, leakage detection), GNN readiness (node features, edge density, connectivity, heterogeneity), and AML typology detectability.

11.4 Extended Evaluators

Additional evaluators: multi-period coherence (opening = prior closing, sequential document IDs), fraud pack effectiveness (detection rates, false positive analysis), OCEL enrichment quality (state transition coverage, resource utilization), and causal intervention magnitude (KPI delta validation, propagation path verification).

11.5 AutoTuner

The AutoTuner closes the loop between evaluation and generation. It analyzes evaluation gaps, generates prioritized `TuningOpportunity` records (Critical, Important, Nice-to-have), and produces a YAML patch for iterative refinement. Supported actions include adjusting transaction volume, distribution parameters, anomaly injection rates, temporal patterns, feature balance, graph connectivity, and correlation matrices.

12 Experiments

We evaluate DataSynth along five dimensions: grounding in real-world data, statistical fidelity, structural coherence, downstream ML utility, and system performance.

12.1 Experimental Setup

Configurations. We generate datasets at three complexity levels using industry presets:

Preset	Complexity	GL Accounts	Entities	Period
Retail	Small	∼100	1	12 months
Manufacturing	Medium	∼400	3	24 months
Financial Svcs.	Large	∼2,500	10	36 months

Each configuration is run with anomaly injection at 2%, 5%, and 10%. All experiments use seed = 42 for reproducibility.

Hardware. AMD Ryzen 9 7950X (16 cores / 32 threads), 64 GB RAM, NVMe SSD, Linux 6.x.

12.2 Grounding in Real-World Accounting Data

To validate that DataSynth produces reference knowledge graphs consistent with real enterprise accounting, we compare against distributions observed in 155 real-world ISO 21378:2019–

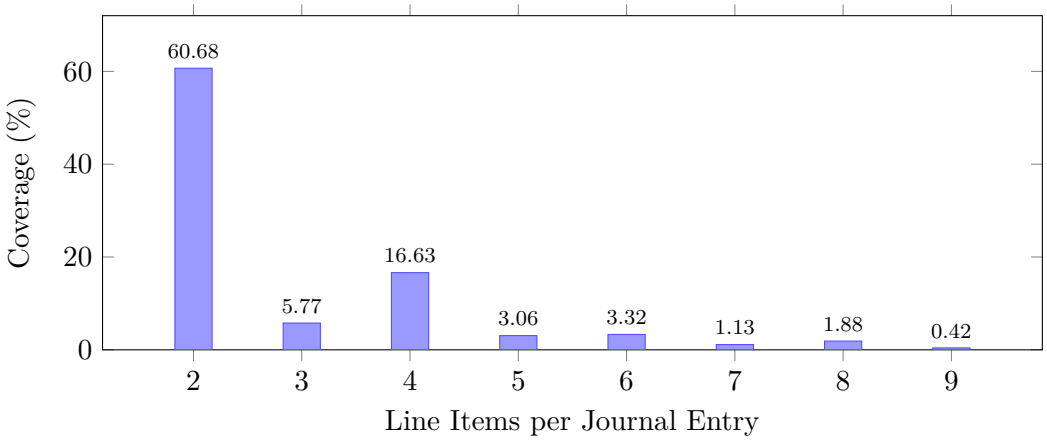

Figure 2: Distribution of journal entry line items (2–9 range, 92.9% of all entries) from 155 real-world datasets [17]. The dominance of 2-line and 4-line entries reflects balanced double-entry bookkeeping—a structural property that DataSynth's reference knowledge graphs reproduce.

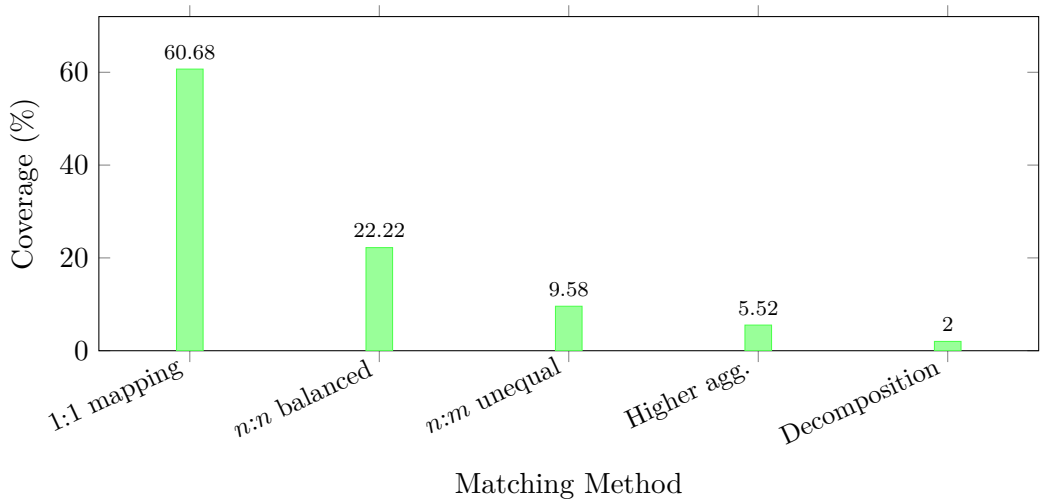

Figure 3: Distribution of journal entry resolution methods for accounting network construction [17]. The 39.3% requiring non-trivial matching motivates DataSynth's forward-generation approach, which eliminates matching ambiguity entirely (Proposition 2.3).

compliant [15] datasets covering all industry sectors, totaling approximately 364 million journal entries with 2.4 billion line items [17].

Journal entry line item distribution. The distribution is extremely heavy-tailed: 92.9% of entries have 2–9 line items, while the maximum observed is 47,059. Figure 2 shows the detailed breakdown within this range. The prominence of 2-line entries (60.7%) and 4-line entries (16.6%) reflects the balanced nature of double-entry bookkeeping.

Graph construction complexity. The combinatorial complexity of accounting network construction is driven by the credit–debit matching problem (Proposition 2.3). In production data, only 60.7% of entries admit trivial 1-to-1 mapping, while 39.3% require more sophisticated strategies (Figure 3). DataSynth's forward generation eliminates this ambiguity entirely—every edge in the reference knowledge graph has known provenance.

Table 6: Benford first-digit MAD scores. Close conformity: MAD < 0.006.

Preset	n **entries**	**MAD**	χ^2 p**-value**
Retail (Small)	52,000	0.0041	0.72
Manufacturing (Med.)	385,000	0.0023	0.91
Financial (Large)	2,100,000	0.0011	0.98

Table 7: Coherence validation for Manufacturing (Medium) preset.

Validator	**Pass Rate**	**Violations**
Balanced entries (debit = credit)	100.0%	0
Accounting equation ($A = L + E$)	100.0%	0
Document chain referential integrity	100.0%	0
Three-way match (within tolerance)	98.7%	52
Subledger = GL control accounts	100.0%	0
IC elimination nets to zero	100.0%	0
Financial statement interconnection	100.0%	0
Trial balance (debits = credits)	100.0%	0

Chart of accounts complexity. The 155 datasets exhibit average 557 financial accounts per entity (median: 367, min: 76, max: 2,536). DataSynth's three complexity tiers span this range, covering the 10th, 50th, and 99th percentiles.

12.3 Statistical Fidelity

Benford's Law Compliance. All scores fall within close conformity. Anderson-Darling tests for log-normal fit yield $p > 0.05$ across all configurations. Correlation preservation: $\hat{\rho} = 0.648 \pm 0.003$ vs. target $\rho = 0.65$ across 10 seeded runs.

12.4 Structural Coherence

The 52 three-way match violations are *intentional*—they correspond to injected price and quantity variances. All hard invariants achieve 100% compliance, confirming the normative knowledge layer is correctly encoded.

The v2.2 capstone validators achieve 100% compliance on the trial balance master proof—confirming that the sum of all generated journal entries from manufacturing, treasury, tax, payroll, and period-close modules reconciles exactly to the closing trial balance. The COGS/WIP reconciliation, interest expense proof, ETR reconciliation, hedge effectiveness check, cash flow reconciliation, equity rollforward, segment-to-consolidated reconciliation, and IC elimination completeness validators likewise pass without exception on deterministic runs.

12.5 Downstream ML Utility

We evaluate whether reference knowledge graphs support effective fraud detection model training on the Financial Services (Large) dataset with 5% anomaly injection.

The GCN model benefits from the relational structure preserved in DataSynth's graph exports. The gap between supervised and unsupervised methods confirms that ground-truth labels provide meaningful signal. When stratified by difficulty score, easy anomalies (difficulty

Table 8: Fraud detection on DataSynth-generated reference knowledge graphs.

Model	F1	AUPRC	R@5%FPR
Isolation Forest	0.42	0.38	0.31
XGBoost	0.81	0.79	0.73
GCN	0.84	0.82	0.78

Table 9: Generation throughput (single-threaded) by record type.

Record Type	Throughput (records/sec)
Journal entries	200,000+
Journal line items	680,000+
Master data (combined)	58,000
Document-flow records	85,000
Subledger records	95,000
Audit FSM events	12,000+

< 0.3) are detected at >95% recall; hard anomalies (difficulty > 0.7) yield ∼45% recall, validating the difficulty scoring.

12.6 System Performance

Throughput scales near-linearly up to 8 cores. At 16 cores, the Manufacturing (Medium) configuration generates a full 24-month dataset (385K journal entries, 1.1M line items, 500K+ document-flow records) in under 30 seconds. Peak resident memory stays below 500 MB even for the Financial Services (Large) configuration.

12.7 Audit Methodology Coverage

Table 10 summarizes the 12 audit methodology blueprints shipped with DataSynth v2.0.0. Each blueprint encodes a complete audit methodology as a finite state machine with procedures, phases, and steps annotated by governing standards and judgment level.

The blueprints enable cross-firm comparison: for example, the three ISA-complete firm-style blueprints share the same procedure count but differ in step granularity, evidence requirements, and the distribution of judgment levels—providing a basis for studying methodology variation in a controlled setting.

12.8 Privacy-Utility Evaluation

Even at High privacy ($\varepsilon = 0.5$), KL divergence remains below 0.02 and Benford MAD stays within close conformity, confirming that fingerprint-calibrated reference knowledge graphs preserve the statistical properties of real enterprise data.

13 Conclusion

We have presented DataSynth, a framework that addresses a fundamental challenge in enterprise audit analytics: the absence of ground truth. We showed that constructing knowledge systems

Table 10: Audit methodology blueprint summary. Judgment level distribution indicates the fraction of steps classified as `data_only` / `ai_assistable` / `human_required`.

Blueprint	Framework	Procedures	Steps	Standards
Generic FSA	ISA	9	24	14
Generic IA	IIA-GIAS	34	82	52
Deloitte FSA	ISA	10	28	16
PwC FSA	ISA	10	28	16
KPMG FSA	ISA	10	28	16
Deloitte ISA	ISA (full)	48	180	34
PwC ISA	ISA (full)	48	180	34
KPMG ISA	ISA (full)	48	180	34
EY GAM Lite	ISA	12	36	18
PCAOB Integrated	PCAOB	42	156	28
SOC 2 Type II	AICPA TSC	20	60	15
Regulatory Exam	Regulatory	16	48	12

Table 11: Fingerprint privacy-utility tradeoff. KL divergence between original and fingerprint-synthesized distributions.

Privacy Level	ε	KL Divergence	Benford MAD
Minimal	5.0	0.003	0.0026
Standard	1.0	0.008	0.0039
High	0.5	0.019	0.0052
Maximum	0.1	0.071	0.0098

primarily from operational enterprise data—while valuable and widely practiced—faces inherent limitations when the source data itself may contain systematic errors, leaving teams without a comprehensive reference against which to verify the resulting knowledge structures. When the source data contains systematic errors, the constructed knowledge may inherit those errors, potentially masking the root causes that audit procedures seek to identify.

Our formal analysis demonstrated three results: **(i)** recovering the true accounting network from observed journal entries is combinatorially infeasible, with the configuration space growing super-exponentially (Proposition 2.3); **(ii)** systematic errors in source data propagate through multi-stage enterprise processes with high probability of remaining undetected (Proposition 2.8); and **(iii)** the inter-dimensional nature of enterprise data—spanning topology, statistics, temporality, norms, and organization—means that errors in one dimension amplify across others (Observation 2.11).

DataSynth addresses this through *forward generation* from a fully specified model, producing *reference knowledge graphs* where the complete audit trail is known by construction across three layers of knowledge: structural (entity types and relationship types covering major enterprise processes), statistical (Benford-compliant distributions, copula-based dependencies), and normative (multiple accounting frameworks, ISA/PCAOB/SOX audit standards, COSO 2013 internal controls).

Experimental evaluation confirms that generated datasets satisfy all hard accounting invariants at 100% compliance, achieve Benford MAD scores below 0.006, and support downstream fraud detection models achieving F1 scores of 0.84 with GNNs. The system provides full prove-

nance for every record in the reference knowledge graph, with generation throughput and domain coverage detailed in Section 12.

13.1 Limitations

- **Domain coverage.** While DataSynth covers 20+ enterprise processes with end-to-end financial coherence—including manufacturing cost accounting (WIP→FG→COGS with standard cost variances), treasury instruments (interest accruals, hedge accounting per ASC 815/IFRS 9), and tax provisions derived from actual GL pre-tax income—specialized domains such as insurance claims, healthcare billing, and government accounting are not yet modeled. XBRL export of financial statements enables compliance testing against regulatory taxonomies. The evaluation suite now includes 25+ coherence validators (up from 15 in v2.0), with 10 additional capstone validators covering COGS/WIP reconciliation, interest expense proof, ETR reconciliation, hedge effectiveness, cash flow reconciliation, equity rollforward, segment-to-consolidated reconciliation, IC elimination completeness, and the trial balance master proof.
- **Learned distributions.** The current parametric approach (log-normal mixtures, copulas) could be complemented by deep generative models for more complex distributional structures, at the cost of reproducibility and interpretability.
- **Cross-enterprise patterns.** The fingerprint module operates on single enterprises. Cross-organization patterns (industry benchmarking, market-level correlations) are not yet modeled.
- **Validation against real knowledge graphs.** While DataSynth's statistical properties are calibrated against 155 real-world datasets, the reference knowledge graph structure has not been validated against actual enterprise knowledge graphs (which are themselves subject to the ground truth problem).

13.2 Future Work

Several of the research directions identified in earlier versions of this paper have been realized: v2.0.0 introduced the audit FSM engine with 12 blueprints spanning ISA, IIA-GIAS, PCAOB, AICPA SOC 2, and Big 4 firm-style methodologies; v2.2.0 adds end-to-end financial coherence across manufacturing cost accounting, treasury instruments, and tax provisions, together with XBRL export and 10 additional capstone coherence validators. The following directions remain open:

1. **Temporal graph network export.** Integrating temporal graph network (TGN) export and the temporal accounting network formalism of Ivertowski [16] for time-evolving reference knowledge graphs with continuous-time event encoding.
2. **Causal inference from synthetic ground truth.** Using DataSynth's fully provenanced datasets—where the causal DAG is known by construction—as benchmarks for evaluating causal discovery algorithms in the enterprise domain.

3. **Federated simulation.** Extending the federated fingerprint aggregation framework to support distributed multi-organization simulation under secure multi-party computation, enabling industry-level reference knowledge graphs without centralizing enterprise data.

4. **Hardware-accelerated graph construction.** Coupling DataSynth's output with GPU-accelerated accounting network generation [17] for interactive audit analytics.

5. **Adaptive anomaly calibration.** Using reinforcement learning to tune anomaly injection such that downstream detector performance matches a target difficulty curve.

6. **Knowledge graph completion benchmarks.** Using DataSynth's reference graphs to create standardized benchmarks for evaluating knowledge graph construction and completion algorithms in the audit domain.

7. **Learned causal transfer functions.** Replacing parametric transfer functions with neural network-based functions fitted from real-world data for more faithful counterfactual propagation.

8. **Judgment automation boundary analysis.** Leveraging the `judgment_level` annotations across 12 methodology blueprints to empirically characterize the automation frontier in audit procedures—identifying which steps are amenable to AI assistance and where human judgment remains indispensable.

13.3 Availability

DataSynth is released as open-source software under the Apache 2.0 license. The source code, documentation, example configurations, and pre-built binaries are available at https://github.com/mivertowski/SyntheticData. A Python wrapper (`datasynth-py`) provides high-level access via `pip install datasynth-py`.

References

[1] Anil Arya, John C. Fellingham, Jonathan C. Glover, Douglas A. Schroeder, and Gilbert Strang. Inferring transactions from financial statements. *Contemporary Accounting Research*, 17:366–385, 2000. doi: 10.1506/L0LW-NX5L-4WUR-9JKL.

[2] Frank Benford. The law of anomalous numbers. *Proceedings of the American Philosophical Society*, 78(4):551–572, 1938.

[3] Daniel J. Bernstein. ChaCha, a variant of Salsa20. In *Workshop Record of SASC*, volume 8, pages 3–5, 2008.

[4] Alessandro Berti, Sebastiaan J. van Zelst, and Wil M.P. van der Aalst. PM4Py: A process mining library for Python. In *ICPM 2019 Demonstration Track*, 2019.

[5] Marcel Boersma. *Complex Networks in Audit: A Data-Driven Modelling Approach*. PhD thesis, Universiteit van Amsterdam, 2024.

[6] Wassim Dbouk and Ibrahim Jamali. Anomaly detection in financial time series by principal component analysis and neural network classifiers. *Algorithms*, 15(10):385, 2022.

[7] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. Calibrating noise to sensitivity in private data analysis. In *Theory of Cryptography Conference (TCC)*, pages 265–284. Springer, 2006.

[8] John Fellingham. The double entry system of accounting. *Accounting, Economics, and Law: A Convivium*, 8(1):20180001, 2018. doi: 10.1515/AEL-2018-0001.

[9] Anahita Farhang Ghahfarokhi, Gyunam Park, Alessandro Berti, and Wil M.P. van der Aalst. OCEL: A standard for object-centric event logs. In *European Conference on Advances in Databases and Information Systems (ADBIS)*, pages 169–175. Springer, 2021.

[10] Ken H. Guo, Xiaoting Yu, and Carla Wilkin. A picture is worth a thousand journal entries: Accounting graph topology for auditing and fraud detection. *Journal of Information Systems*, 36(2):53–81, 2022. doi: 10.2308/ISYS-2021-003.

[11] Waleed Hilal, S. Andrew Gadsden, and John Yawney. Financial fraud: A review of anomaly detection techniques and recent advances. *Expert Systems with Applications*, 193:116429, 2022.

[12] IEEE Computational Intelligence Society. IEEE-CIS fraud detection dataset. Kaggle Competition, 2019.

[13] Yuji Ijiri. On the generalized inverse of an incidence matrix. *Journal of the Society for Industrial and Applied Mathematics*, 13:827–836, 1965. doi: 10.1137/0113053.

[14] Institute of Internal Auditors. Global internal audit standards. Technical report, The Institute of Internal Auditors, 2024. URL https://www.theiia.org/en/standards/.

[15] International Organization for Standardization. ISO 21378:2019 — audit data collection. International Standard, 2019.

[16] Michael Ivertowski. Empowering financial analysis: A dynamic system for merging temporal accounting and OCPM networks. Unpublished, 2024.

[17] Michael Ivertowski, Mouritz Barnard, and Ina Braun. Hardware accelerated method for accounting network generation. In *Preprint*, 2024. Ernst & Young Ltd., Zurich.

[18] James Jordon, Jinsung Yoon, and Mihaela van der Schaar. PATE-GAN: Generating synthetic data with differential privacy guarantees. In *International Conference on Learning Representations (ICLR)*, 2019.

[19] Akim Kotelnikov, Dmitry Baranchuk, Ivan Rubachev, and Artem Babenko. TabDDPM: Modelling tabular data with diffusion models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, pages 17564–17579, 2023.

[20] Pierre Jinghong Liang, Andi Wang, Leman Akoglu, and Christos Faloutsos. Pattern recognition and anomaly detection in bookkeeping data. Carnegie Mellon University, 2021.

[21] Edgar Alonso Lopez-Rojas and Stefan Axelsson. BankSim: A bank payments simulator for fraud detection research. In *26th European Modeling and Simulation Symposium (EMSS)*, 2014.

[22] Roger B. Nelsen. *An Introduction to Copulas*. Springer, 2nd edition, 2006.

[23] Mark J. Nigrini. *Benford's Law: Applications for Forensic Accounting, Auditing, and Fraud Detection*. John Wiley & Sons, 2012.

[24] Neha Patki, Roy Wedge, and Kalyan Veeramachaneni. The synthetic data vault. In *IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, pages 399–410, 2016.

[25] Tahereh Pourhabibi, Kok-Leong Ong, Booi H. Kam, and Yee Ling Boo. Fraud detection: A systematic literature review of graph-based anomaly detection approaches. *Decision Support Systems*, 133:113303, 2020.

[26] Marco Schreyer, Timur Sattarov, Damian Borth, Andreas Dengel, and Bernd Reimer. Detection of anomalies in large scale accounting data using deep autoencoder networks. In *arXiv preprint arXiv:1709.05254*, 2017.

[27] Marco Schreyer, Timur Sattarov, and Damian Borth. Federated and privacy-preserving learning of accounting data in financial statement audits. In *Proceedings of the Third ACM International Conference on AI in Finance (ICAIF)*, pages 105–113, 2022.

[28] Aivin V. Solatorio and Olivier Dupriez. REaLTabFormer: Generating realistic relational and tabular data using transformers. *arXiv preprint arXiv:2302.02041*, 2023.

[29] Miklos A. Vasarhelyi, Michael G. Alles, and Alexander Kogan. Principles of analytic monitoring for continuous assurance. *Journal of Emerging Technologies in Accounting*, 1(1): 1–21, 2004.

[30] Jarrod West and Maumita Bhattacharya. Intelligent financial fraud detection: A comprehensive review. *Computers & Security*, 57:47–66, 2016.

[31] Liyang Xie, Kaixiang Lin, Shu Wang, Fei Wang, and Jiayu Zhou. Differentially private generative adversarial network. In *arXiv preprint arXiv:1802.06739*, 2018.

[32] Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. Modeling tabular data using conditional GAN. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

A Anomaly Type Catalog

Table 12 lists the primary anomaly types used in the default injection framework. The codebase includes over 130 fine-grained subtypes across the five categories.

Table 12: Primary anomaly types with default injection parameters.

Category	Type	Weight	Amount Range	Conditional
Fraud	FictitiousEntry	1.0	\$10K–500K	No
	RoundDollarManipulation	2.0	\$1K–100K	No
	JustBelowThreshold	2.5	Threshold	Yes
	SelfApproval	1.5	—	Yes
	ExceededApprovalLimit	1.5	—	Yes
	SegregationOfDutiesViolation	1.0	—	Yes
	DuplicatePayment	2.0	\$5K–200K	No
	FictitiousVendor	0.5	\$25K–1M	No
	RevenueManipulation	0.5	\$100K–5M	No
	ImproperCapitalization	1.0	\$10K–500K	No
Error	DuplicateEntry	2.0	Original	No
	ReversedAmount	1.5	Original	No
	WrongPeriod	1.5	Original	No
	WrongAccount	1.5	Original	No
	MissingReference	2.0	—	No
Process	LatePosting	2.0	—	No
	SkippedApproval	1.0	—	Yes
	ThresholdManipulation	1.5	Threshold	Yes
Statistical	UnusualAmount	2.0	$> 3\sigma$	No
	TrendBreak	1.0	—	No
	BenfordViolation	1.0	Anti-Benford	No
Relational	CircularTransaction	0.5	\$50K–2M	No
	DormantAccountActivity	1.0	\$5K–100K	No

B Output File Inventory

Table 13 lists the primary output files produced by a full DataSynth generation run.

C Copula Sampling Algorithms

Algorithm 2: Clayton Copula Bivariate Sampling

Input: Parameter $\theta > 0$, RNG seed

Output: $(u, v) \in [0, 1]^2$ with Clayton dependence

$u \leftarrow \text{Uniform}(0, 1)$;

$t \leftarrow \text{Uniform}(0, 1)$;

$v \leftarrow \left((u^{-\theta} - 1) \cdot t^{-\theta/(\theta+1)} + 1\right)^{-1/\theta}$;

return (u, v);

Algorithm 3: Gumbel Copula Bivariate Sampling (Marshall-Olkin)

Input: Parameter $\theta \geq 1$, RNG seed

Output: $(u, v) \in [0, 1]^2$ with Gumbel dependence

$S \leftarrow \text{PositiveStable}(1/\theta)$;

$E_1, E_2 \leftarrow \text{Exponential}(1), \text{Exponential}(1)$;

$u \leftarrow \exp(-(E_1/S)^{1/\theta})$;

$v \leftarrow \exp(-(E_2/S)^{1/\theta})$;

return (u, v);

Algorithm 4: Student-t Copula Bivariate Sampling

Input: Correlation ρ, degrees of freedom ν, RNG seed

Output: $(u, v) \in [0, 1]^2$ with Student-t dependence

$w_1, w_2 \leftarrow \mathcal{N}(0, 1), \mathcal{N}(0, 1)$;

$z_1 \leftarrow w_1$;

$z_2 \leftarrow \rho \cdot w_1 + \sqrt{1 - \rho^2} \cdot w_2$;

$S \leftarrow \chi^2_\nu / \nu$;

$t_1 \leftarrow z_1 / \sqrt{S}$;

$t_2 \leftarrow z_2 / \sqrt{S}$;

$u \leftarrow F_{t,\nu}(t_1)$;

$v \leftarrow F_{t,\nu}(t_2)$;

return (u, v);

D COSO Framework Integration

DataSynth implements the COSO 2013 Internal Control—Integrated Framework with five components and 17 principles:

#	Component	Principles
1	Control Environment	1–5
2	Risk Assessment	6–9
3	Control Activities	10–12
4	Information & Communication	13–15
5	Monitoring Activities	16–17

Generated internal controls include 12 transaction-level controls (C001–C060) and 6 entity-level controls (C070–C081), each mapped to COSO components, principles, and control scopes. Maturity levels (Non-Existent, Ad Hoc, Repeatable, Defined, Managed, Optimized) are assigned per control.

E Configuration Schema Reference

The complete YAML configuration schema comprises 30+ sections.

Table 14: Top-level configuration sections.

Section	Purpose
`global`	Industry, period, seed, complexity
`companies`	Multi-entity setup, consolidation groups
`chart_of_accounts`	CoA complexity, GL structure
`transactions`	JE volume, process distribution
`document_flows`	P2P / O2C parameters, matching tolerances
`distributions`	Benford, mixtures, copulas, drift
`temporal_patterns`	Business days, period-end, processing lags
`anomaly_injection`	Fraud rates, strategies, severity targets
`internal_controls`	COSO maturity, SoD rules
`intercompany`	IC matching, elimination rules
`banking`	KYC/AML, customer profiles, typologies
`accounting_standards`	IFRS / US GAAP / French / German GAAP
`audit_standards`	ISA / PCAOB compliance level
`audit`	Engagement, workpapers, evidence
`vendor_network`	Supply chain tiers, concentration limits
`customer_segmentation`	Value segments, lifecycle stages
`hr`	Payroll, time & attendance, expenses
`manufacturing`	Production, quality, inventory
`tax`	Jurisdictions, provisions, transfer pricing
`treasury`	Cash, hedging, debt, covenants
`esg`	Environmental, social, governance
`project_accounting`	WBS, earned value, milestones
`financial_reporting`	Statements, KPIs, budgets, segments
`sales_quotes`	Quote-to-order pipeline
`graph_export`	PyG / Neo4j / DGL output options
`output`	Format, compression, sink

F Knowledge Dimension Coverage per Entity Type

Table 15 maps representative entity types to the knowledge dimensions they primarily encode (Definition 2.10).

Table 15: Knowledge dimension coverage for representative entity types. T = Topological, S = Statistical, τ = Temporal, N = Normative, O = Organizational.

Entity Type	**T**	**S**	$\boldsymbol{\tau}$	**N**	**O**
JournalEntry	✓	✓	✓	✓	✓
PurchaseOrder	✓	✓	✓	✓	✓
Vendor	✓	–	–	✓	✓
InternalControl	✓	–	–	✓	✓
TaxProvision	–	✓	✓	✓	–
EmissionRecord	–	✓	✓	✓	–
HedgingInstrument	✓	✓	✓	✓	–
AuditOpinion	✓	–	✓	✓	✓
ProjectCostLine	✓	✓	✓	–	✓
AnomalyLabel	✓	✓	✓	✓	✓

Table 13: Output file inventory by category.

Category	Files
Transactions	journal_entries, acdoca
Master Data	vendors, customers, materials, fixed_assets, employees, cost_centers
Document Flow	purchase_orders, goods_receipts, vendor_invoices, payments, sales_orders, deliveries, customer_invoices, customer_receipts, document_references
Sourcing	sourcing_projects, supplier_qualifications, rfx_events, supplier_bids, bid_evaluations, procurement_contracts, catalog_items, supplier_scorecards
HR / Payroll	payroll_runs, payslips, time_entries, expense_reports, benefit_enrollments
Manufacturing	production_orders, routing_operations, quality_inspections, cycle_counts, bom_components, inventory_movements
Financial	balance_sheet, income_statement, cash_flow_statement, changes_in_equity, financial_kpis, budget_variance
Tax	tax_jurisdictions, tax_codes, tax_returns, tax_provisions, deferred_tax_rollforward, withholding_tax
Treasury	cash_positions, cash_forecasts, cash_pools, hedging_instruments, debt_instruments, debt_covenants
ESG	emission_records, energy_consumption, water_usage, waste_records, diversity_metrics, safety_incidents
Project	projects, project_cost_lines, earned_value_metrics, change_orders, project_milestones
Sales	sales_quotes, sales_quote_items
Subledgers	ar_*, ap_*, fa_*, inventory_*
Period Close	trial_balances/, accruals, depreciation, closing_entries
Consolidation	eliminations, currency_translation, consolidated_trial_balance
Labels	anomaly_labels, fraud_labels, quality_issues, quality_labels
Controls	internal_controls, coso_control_mapping, sod_*
Process Mining	event_log.json (OCEL 2.0), objects.json, process_variants
Banking	banking_customers, bank_transactions, kyc_profiles, aml_typology_labels, bank_reconciliations
Audit	audit_engagements, audit_workpapers, audit_evidence, audit_risks, audit_findings, audit_opinions, sox_assessments
Standards	customer_contracts, performance_obligations, leases, rou_assets, lease_liabilities, fair_value_measurements, impairment_tests, isa_mappings, confirmations
Graph	transaction_graph.*, approval_network.*, entity_graph.* (PyG/Neo4j/DGL)
XBRL	financial_reporting/xbrl/ — XBRL 2.1 instance documents (US GAAP / IFRS taxonomies)
Counterfactual	scenario_manifest, baseline/, counterfactual/, scenario_diff
Sessions	session_state.dss, period_manifest